

Salvatore D'Angelo



Build Your First Website

with HTML & CSS

Step by step, from a blank page to a complete, responsive site

HTML5

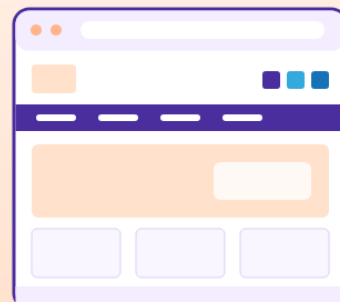


CSS3



```
index.html

<!DOCTYPE html>
<html>
  <body>
    <h1>Hello!</h1>
    <nav>...</nav>
  </body>
</html>
```



Structure
with HTML



Style
with CSS



Responsive
Layouts



Forms &
Gallery



From a blank page to a real website.
Your journey starts here.

To every beginner who has ever typed

`<h1>Hello, World!</h1>`

into a plain text file and watched it come alive in a browser.

Contents

Preface	6
Who This Book Is For	6
How This Book Is Structured	6
How to Read the Code	7
Source Code and Updates	7
1 Getting Started with HTML: A Beginner's Guide	8
1.1 Introduction	8
1.2 What is HTML?	8
1.3 A bit of History	9
1.4 The HTML Basic structure	11
1.5 HTML Elements, Tags, and Attributes	12
1.6 The head tag	13
1.7 The body tag	14
1.8 Headlines	14
1.9 Paragraphs	16
1.10 An example of an HTML document	16
1.11 Lorem Ipsum placeholder text	17
1.12 Organizing Content with HTML Sections	17
1.13 Text Formatting with bold, italic, and underline	19
1.14 Creating Hyperlinks in HTML	20
1.15 Creating Lists with HTML: Ordered and Unordered Lists	21
1.16 Enhancing Your Webpage with Images: The Tag	22
1.17 Conclusion	24
2 Mastering the Basics of CSS: A Beginner's Guide	26
2.1 Introduction	26
2.2 What is CSS?	26
2.3 CSS Rules and Declarations	28

2.4	CSS Inheritance	29
2.5	CSS Selectors and Cascading: The Building Blocks of Web Design	30
2.6	The Box Model	32
2.7	Layout in CSS	34
2.8	Typography in CSS	36
2.9	Color and Background in CSS	38
2.10	Transitions and Animations in CSS	40
2.11	Responsive Design in CSS	42
2.12	How to Include CSS in a web page	43
2.13	Applying CSS Principles to Create a Website Style	44
2.14	Conclusion	47
3	Building a Professional Website Layout with CSS: Header, Navigation, and Responsive Design	49
3.1	Introduction	49
3.2	Your First CSS File	50
3.3	Publishing Your Website	51
3.4	Header and Footer with Semantic HTML	51
3.5	Social Media Icons with Font Awesome	53
3.6	Completing the Footer	54
3.7	The Navigation Bar	54
3.8	Building the Home Page	56
3.8.1	The Hero Area	56
3.8.2	The Newsletter Section	56
3.9	Making the Site Responsive	57
3.9.1	The Viewport Meta Tag	57
3.9.2	Media Queries	58
3.9.3	The Hamburger Menu	59
3.10	The Final File Structure	60
3.11	Conclusion	61
4	Building the Complete Website: Real Pages, HTML Forms, and a Photo Gallery	62
4.1	Introduction	62
4.2	A New CSS File: page.css	63
4.3	The About Me Page	64
4.4	The Start Here Page and <figure>	65

4.5	The Resources Page	66
4.6	The Contacts Page and HTML Forms	66
4.7	The Gallery Page: CSS Grid and JavaScript	69
4.7.1	CSS Grid for the Image Grid	69
4.7.2	The JavaScript Lightbox	70
4.8	The Final File Structure	72
4.9	Conclusion	72
Glossary		74
What's Next?		76
	Where to Go From Here	76
	JavaScript	76
	Modern CSS	76
	Accessibility	77
	Static Site Generators and Hosting	77
	Stay in Touch	77

Preface

Every web page you have ever visited is built on two languages: HTML, which gives a page its structure, and CSS, which gives it its look. Frameworks, page builders, and JavaScript libraries come and go, but underneath all of them there is always plain HTML and CSS.

Most beginner tutorials stop at isolated snippets: a heading here, a coloured box there. You learn the tags but never see how they come together into a real site.

This book takes a different path. It follows a single project, **html-hero**, from a blank text file to a complete, responsive, multi-page website. Every chapter introduces a handful of concepts and immediately applies them to that project, so by the last page you will have built a real site with a header, a navigation bar, a home page with a hero image, a contact form, and an interactive photo gallery.

Who This Book Is For

This book is for readers who:

- Have never written a line of HTML or CSS and want to start from zero
- Have copied snippets from the web but never understood how a page is structured
- Want to build their own personal website without relying on a page builder
- Want a single, coherent reference instead of scattered blog posts

No prior programming knowledge is assumed. A text editor and a web browser are all you need.

How This Book Is Structured

Each chapter builds on the previous one, and together they follow the html-hero project from start to finish:

1. **Getting Started with HTML: A Beginner's Guide** — Learn HTML from scratch — build a real five-page website with headings, paragraphs, links, lists, and images, step by

step.

2. **Mastering the Basics of CSS: A Beginner's Guide** — Master the core CSS concepts — selectors, the box model, layout, typography, colours, animations, and responsive design — and apply them to a real website.
3. **Building a Professional Website Layout with CSS: Header, Navigation, and Responsive Design** — Turn your plain HTML site into a professional layout — styled header, footer, navigation bar, home page with hero image, and a mobile-friendly hamburger menu.
4. **Building the Complete Website: Real Pages, HTML Forms, and a Photo Gallery** — Build the real inner pages of your site — About Me, Start Here, Resources, a contact form, and an interactive photo gallery with CSS Grid and JavaScript.

How to Read the Code

All the source code is organised in lessons. Each lesson has its own folder, so you can open the result of every step in your browser and compare it with your own work. When a chapter says “download the file of lesson 5”, you will find it in the corresponding `lesson-05` folder of the repository.

Source Code and Updates

The examples in this book are maintained on GitHub at github.com/sasadangelo/html-hero.

For corrections, suggestions, or feedback, write to sasadangelo@gmail.com or visit code4projects.org.

Chapter 1

Getting Started with HTML: A Beginner's Guide

1.1 Introduction

HTML is the language that forms the foundation of the web. By the end of this guide you will have built a five-page website with paragraphs, headlines, links, images, and formatted text — entirely from scratch.

You can find the source code of this guide in the [part-1 folder of the html-hero repository](#). This guide covers the first 10 lessons.

You should read this article if:

- you have never written a line of HTML and want to start with a real project
- you want to understand how a browser renders a web page, from the DOCTYPE down
- you want a hands-on foundation before moving to CSS and page layout

1.2 What is HTML?

Hypertext Markup Language (HTML) is the language used for creating web pages and applications. It allows developers to create web pages with paragraphs, headings, links, images, video, and more.

HTML uses a series of Elements each one composed of two Tags. Tags are enclosed in angle brackets and typically come in pairs, with an opening and closing tag. The content goes between them. For example, the opening tag `<p>` represents a paragraph, and the closing tag `</p>` indicates the end of the paragraph.



Figure 1.1: Getting Started with HTML: A Beginner's Guide

By using HTML, developers can create rich, interactive, and engaging web pages that can be accessed by users across the world.

1.3 A bit of History

HTML was initially developed to provide a way to share scientific research and information over the internet. As the World Wide Web grew in popularity, the need for a more advanced and user-friendly language for creating web pages became evident. Over the years, HTML has undergone several revisions and updates to add new features and capabilities.

Today, HTML is the backbone of the World Wide Web, and almost all websites and web applications use HTML in some form or another. It is a powerful and versatile language that continues to evolve and improve, providing new and exciting ways for developers to create dynamic and interactive web content.

Over the years, different HTML versions were released. Here is a brief overview of these versions and the key features and improvements they introduced:

- **HTML 1.0:** This was the first version of HTML, developed in 1990 by Tim Berners-Lee. It provided a basic set of tags for creating simple web pages, including tags for headings, paragraphs, lists, and links.
- **HTML 2.0:** This version of HTML was released in 1995 and added support for tables, text formatting, and additional multimedia elements like images and audio files.

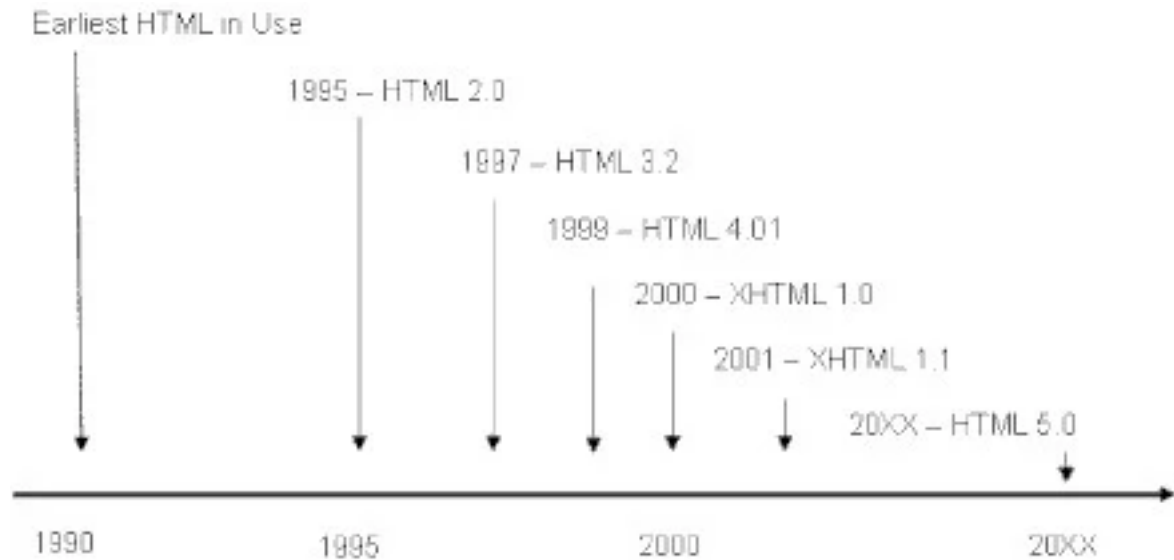


Figure 1.2: HTML History

- **HTML 3.2:** The third version of HTML, released in 1997, introduced support for forms, scripts, and other advanced features. It also added support for style sheets, which allowed developers to separate the presentation of a web page from its content.
- **HTML 4.01:** This version of HTML, released in 1999, added support for more multimedia elements and introduced new tags for creating interactive web applications. It also added support for internationalization and the ability to use XML-based languages like MathML and SVG.
- **HTML 5:** The latest version of HTML, released in 2014, added many new features and capabilities, including support for video and audio playback, local storage, and offline web applications. It also introduced a new approach to web development called “responsive design”, which allows web pages to adapt to different screen sizes and devices.

The history of HTML, however, has not always been linear. At a certain point, the W3C competition decided to embark on a new path which, over time, has proved unsuccessful. This path involved the development of XHTML as a natural succession of HTML 4.0.

XHTML, which stands for Extensible Hypertext Markup Language, is a markup language that is based on HTML but uses stricter syntax rules. It was developed in the late 1990s as a way to make HTML more consistent and compatible with other XML-based languages.

- **XHTML 1.0**, the first version of XHTML, was released in 2000 and was based on HTML 4.0. It introduced several changes to the HTML language, including the use of lowercase

tags, the requirement for all tags to be closed, and the requirement to use quotation marks around attribute values.

- **XHTML 1.1**, released in 2001, added support for additional modules, such as ruby annotations and bidirectional text, and introduced additional rules for creating well-formed documents.
- **XHTML 2.0**, which was under development for several years but was never completed or released, would have introduced even more significant changes to the HTML language. However, this version of XHTML was eventually abandoned in favor of a new approach called HTML5.

Today, XHTML is not as widely used as it once was, but it continues to be supported by many web browsers and is still used by some developers for creating web pages.

1.4 The HTML Basic structure

A basic HTML document has a very simple structure and consists of the following elements:

- A **DOCTYPE** declaration specifies the version of HTML that the document uses.
- The **html** element is the root element of the document and encloses all other elements.
- A **head** element contains metadata about the document, such as the title, character set, styles, and scripts.
- A **body** element contains the main content of the document, such as headings, paragraphs, images, and links.

Here is an example of a basic HTML document:

```
<!DOCTYPE html>
<html>
<head>
  <title>My Document Title</title>
</head>
<body>
  <h1>Hello, World!</h1>
  <p>This is my first web page.</p>
</body>
</html>
```

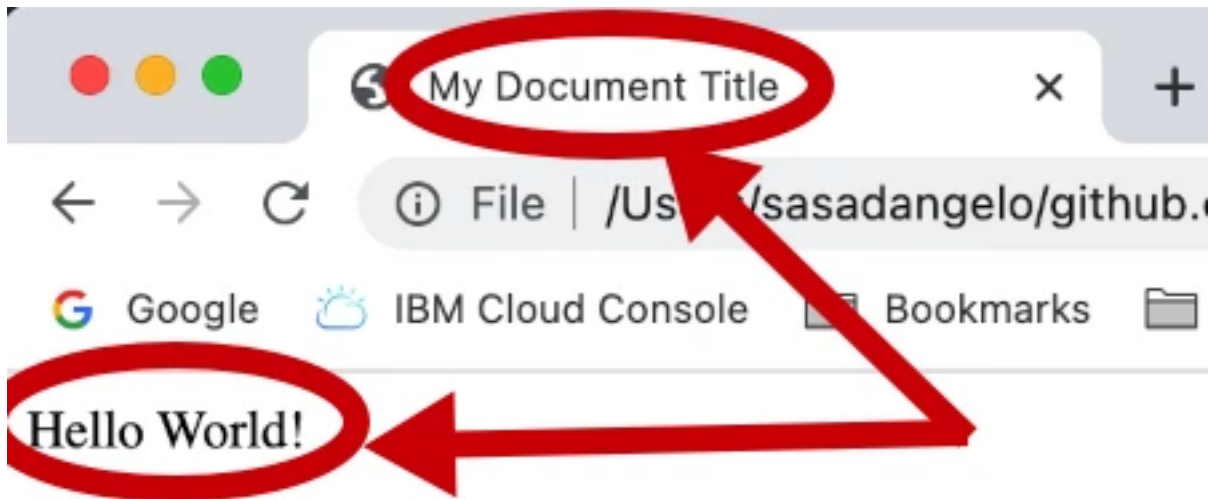


Figure 1.3: My First HTML File

As you can see, the DOCTYPE declaration, the HTML element, the head element, and the body element are all nested within each other, with the DOCTYPE declaration at the top, and the body element at the bottom. The head element contains metadata about the document, and the body element contains the main content.

Download your first HTML document of [lesson 1](#). Its name is index.html and you can open it with your browser (Chrome, Firefox, Microsoft Edge, Internet Explorer, or Safari) by clicking on File->Open File... to render it. Here is the result.

1.5 HTML Elements, Tags, and Attributes

In HTML, elements are the basic building blocks used to create a web page. These elements are represented by tags, which are keywords surrounded by angle brackets. For example, the p tag is used to create a paragraph element, and the h1 tag is used to create a heading element.

Here is an example of a simple HTML tag:

```
<p>This is a paragraph.</p>
```

In this example, the <p> tag is used to mark up a paragraph of text. The <p> tag has an opening tag <p> and a closing tag </p>, with the content in between.

In addition to the basic structure provided by tags, HTML also supports the use of attributes. Attributes are used to provide additional information about an element. They are added to the opening tag of an element and consist of a name and a value, separated by an equals sign. For

example, the `src` attribute is used to specify the source of an image, and the `href` attribute is used to specify a link.

Here is an example of an HTML tag with an attribute:

```
<a href="http://www.example.com">This is a link.</a>
```

In this example, the `<a>` tag is used to create a link to another web page. The `href` attribute is used to specify the URL of the page that the link should go to.

So, to sum up, Elements are used to structure and format the content of a web page. Each Element is enclosed in an open and closed Tag. Attributes are used to provide additional information or to modify the behavior of a tag. Together, elements, tags and attributes form the basic building blocks of an HTML document.

1.6 The head tag

The `<head>` element is a container for HTML metadata (data about the document, such as its title, character set, styles, and scripts). The metadata is not displayed on the page but is used by browsers and search engines to understand the content of the page.

The `<head>` element should be placed at the beginning of the `<html>` element and must contain a `<title>` element, which specifies the title of the document. The `<head>` element can also include other elements, such as `<style>` for styling the page with CSS, `<link>` for linking to external resources, and `<meta>` for defining metadata, such as the character set, keywords, and description of the page.

Here is an example of an `<head>` element that includes a `<title>` element and a `<meta>` element:

```
<head>
  <meta charset="utf-8">
  <title>My Document Title</title>
</head>
```

As you can see, the `<title>` element appears within the `<head>` element and specifies the title of the page, and the `<meta>` element appears within the `<head>` element and defines the character set of the page.

1.7 The body tag

The `<body>` element is a container for the main content of an HTML. The content of the `<body>` element is displayed in the main window or viewport of the browser.

The `<body>` element should be placed after the `<head>` element and can contain any type of content, such as headings, paragraphs, lists, images, links, and more. This content is rendered by the browser according to the styles and layout defined by the document or by external stylesheets.

Here is an example of a `<body>` element that contains a heading and a paragraph:

```
<body>
  <h1>Hello World!</h1>
  <p>This is my first web page.</p>
</body>
```

As you can see, the `<h1>` element and the `<p>` element are nested within the `<body>` element, and their content is displayed on the page. The `<h1>` element represents a heading, and the `<p>` element represents a paragraph.

1.8 Headlines

In HTML, a headline is a piece of text that is used to indicate the importance or relevance of the content that follows. Headlines are typically larger and bolder than the surrounding text, and they may also be styled differently to make them stand out. There are several different levels of headlines in HTML, ranging from level 1 (the most important) to level 6 (the least important). The level of a headline is indicated by the use of a specific HTML tag, such as `<h1>` for a level 1 headline or `<h2>` for a level 2 headline. For example, the following code would create a level 2 headline that says “Introduction”:

```
<h2>Introduction</h2>
```

Headlines are used to organize and structure the content of a webpage, making it easier for readers to navigate and understand the information being presented. They also help search engines understand the hierarchy and organization of a webpage, which can improve its visibility and ranking in search results.

Download the **index.html** file of [lesson 2](#) and open it with your browser. Here is the result.

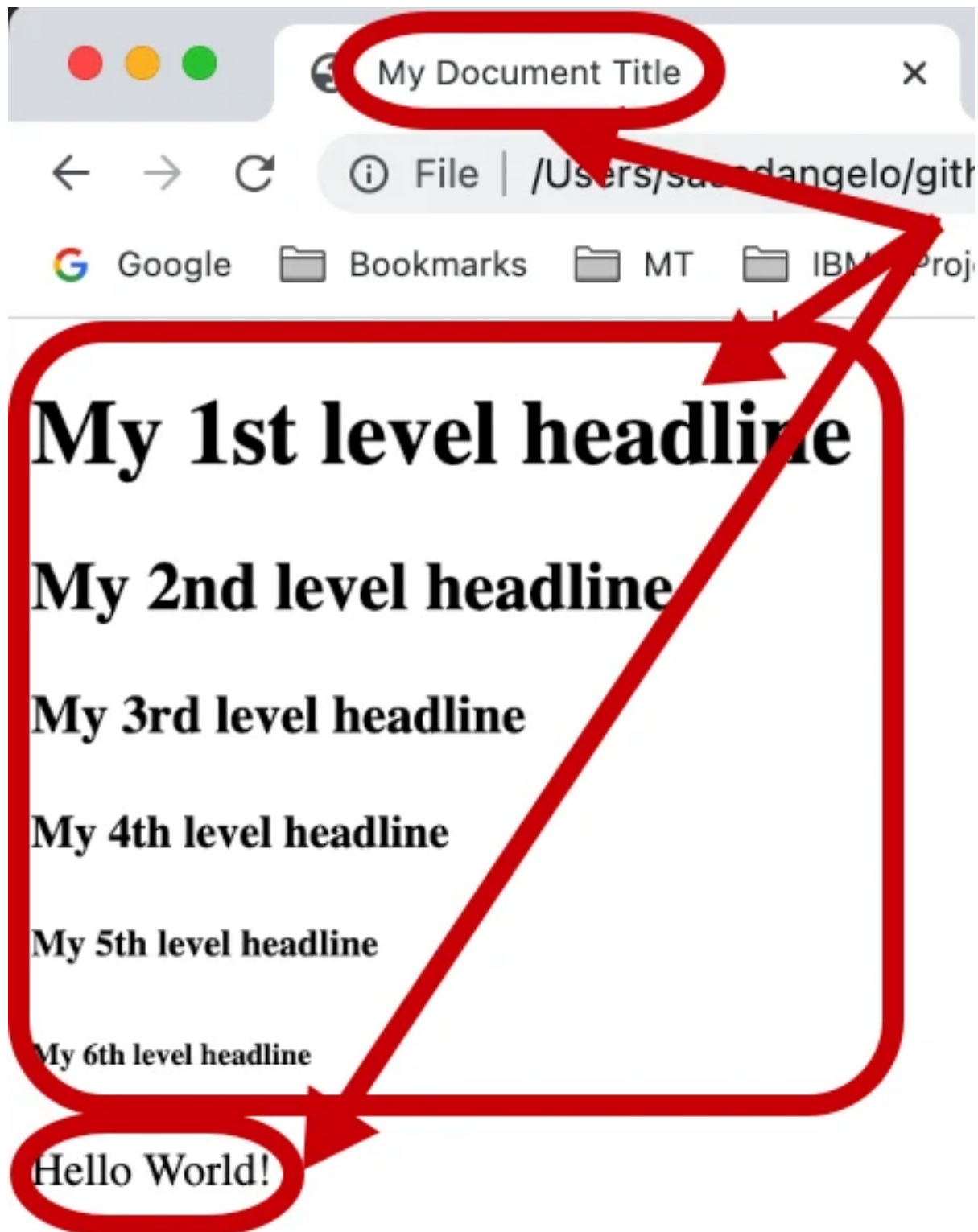


Figure 1.4: HTML Headlines

1.9 Paragraphs

In HTML, the `<p>` tag is used to define a paragraph of text. The `<p>` tag tells the web browser that the enclosed text should be treated as a paragraph, and it automatically adds some extra space before and after the paragraph to help separate it from other content on the page. The `<p>` tag is an example of an “inline” element, which means that it can be used within a block-level element such as a `<div>` or a `<section>` to add structure and organization to the content of a webpage. Here is an example of how the `<p>` tag might be used:

```
<p>This is the first paragraph of text on the page.</p>
<p>This is the second paragraph of text on the page.</p>
```

In the example above, the first and second paragraphs of text are each enclosed within their own `<p>` tags. This tells the web browser to treat each paragraph as a distinct block of text and to add the appropriate amount of space between them. The `<p>` tag is a very common and useful element in HTML, and it is used on almost every webpage to structure the content and make it easier to read.

1.10 An example of an HTML document

Here is an example of how you might use headlines and paragraphs together in an HTML document:

```
<h1>Introduction</h1>
<p>This is the first paragraph of the introduction, which provides a brief overview
↪ of the topic.</p>
<p>This is the second paragraph of the introduction, which expands on the ideas
↪ introduced in the first paragraph.</p>
<h2>Background Information</h2>
<p>This is the first paragraph of the background section, which provides additional
↪ context and detail about the topic.</p>
<p>This is the second paragraph of the background section, which continues to build
↪ on the information from the first paragraph.</p>
<h3>Key Points</h3>
<p>This is the first paragraph of the key points section, which summarizes the most
↪ important information from the article.</p>
<p>This is the second paragraph of the key points section, which provides a concise
↪ overview of the key ideas discussed in the article.</p>
```

In this example, the `<h1>` tag is used to create a level 1 headline for the introduction, the `<h2>` tag is used to create a level 2 headline for the background information, and the `<h3>` tag is used to create a level 3 headline for the key points. Each of these headlines is followed by one or more paragraphs of text, which provide more details and information about the topic. The use of headlines and paragraphs helps to organize the content of the webpage and make it easier for readers to understand and navigate.

1.11 Lorem Ipsum placeholder text

Lorem Ipsum is a placeholder text that is often used by designers and printers to fill in space in a document or website. It is a dummy text that is used to give an idea of what the final content will look like, without actually having to come up with real content. At the [following link](#), you can use an online generator to create dummy text for your website.

Download the `index.html` file of [lesson 3](#) and open it with your browser. Here is the result.

1.12 Organizing Content with HTML Sections

A section in HTML refers to a distinct part of a web page that can be defined by a headline and one or more paragraphs. These sections can be used to organize and divide content on a web page into logical groups, making it easier for users to understand and navigate the page.

For example, a web page about a news article can have multiple sections: one for the introduction, one for the main body of the article, and another for the conclusion. Each section would have its headline, which can be created using one of the heading elements (`<h1>` to `<h6>`), and one or more paragraphs, which can be created using the `<p>` element.

```
<h1>My Article Title</h1>
<p>This is the first paragraph of the introduction, which provides a brief overview
↪ of the topic.</p>
<p>This is the second paragraph of the introduction, which expands on the ideas
↪ introduced in the first paragraph.</p>
<h2>My 1st section headline</h2>
<p>This is the first paragraph of the 1st section, which provides additional context
↪ and detail about the topic.</p>
<p>This is the second paragraph of the 1st section, which continues to build on the
↪ information from the first paragraph.</p>
<h2>My 2nd section headline</h2>
```

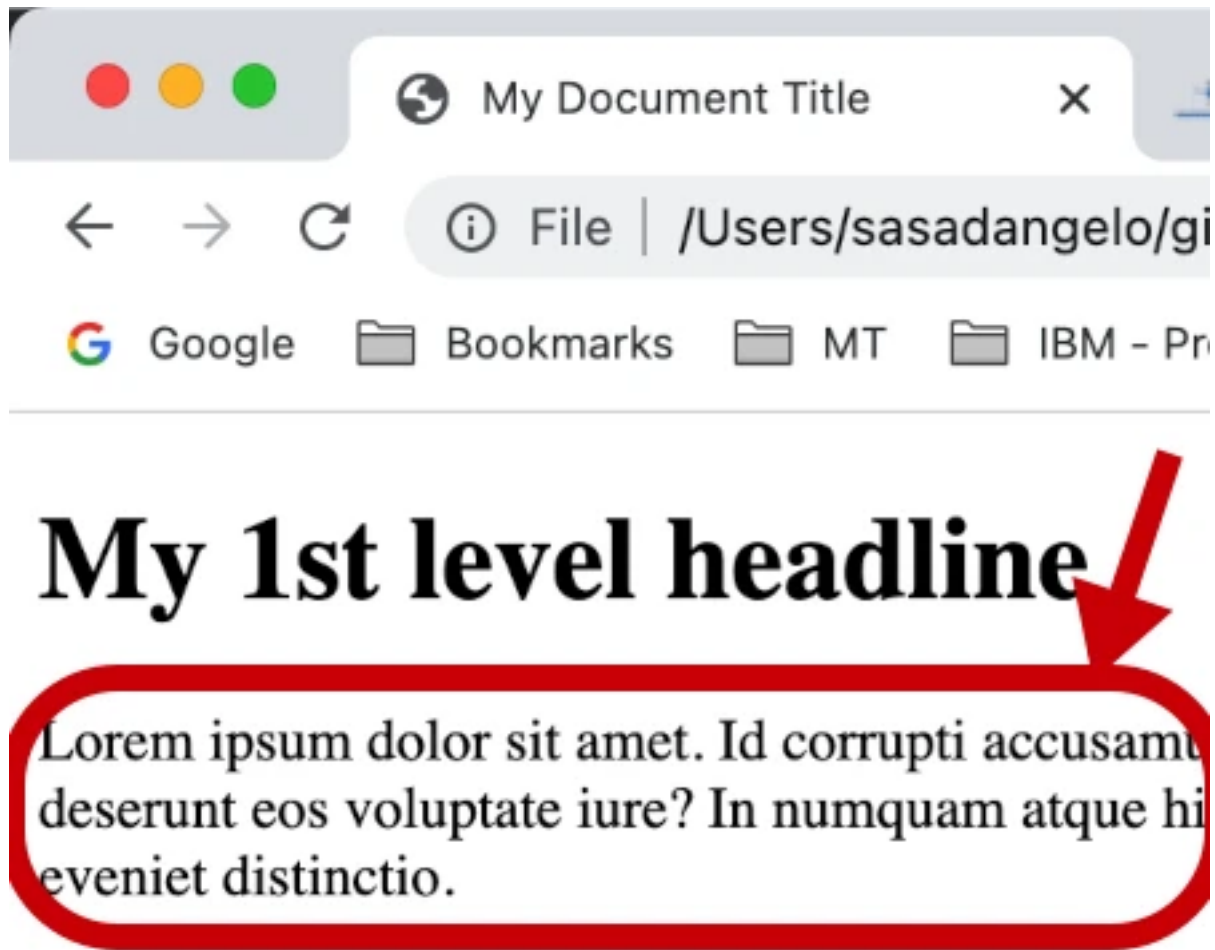


Figure 1.5: HTML Paragraph

```
<p>This is the first paragraph of the 2nd section, which summarizes the most  
↳ important information from the article.</p>  
<p>This is the second paragraph of the 2nd section, which provides a concise overview  
↳ of the key ideas discussed in the article.</p>
```

It's worth noting that before HTML 5, there were no specific tags to create sections. In this guide, I just showed a way of organizing the content in a logical way. You can use any tags that you think will be more appropriate to create sections.

Download the `index.html` file of [lesson 4](#), [lesson 5](#), and [lesson 6](#) and open them with your browser. You will see a complete web page built progressively, section by section, where each section contains a headline and one or more paragraphs.

1.13 Text Formatting with bold, italic, and underline

HTML supports several ways to format text, including bold, italic, and underline. Here's how to use each one. To make text bold, you can use the `<strong>` or `<b>` tag. For example:

```
<p>This text is <strong>bold</strong></p>
```

To make text italic, you can use the `<em>` or `<i>` tag. For example:

```
<p>This text is <em>italic</em></p>
```

To underline text, you can use the `<u>` tag. For example:

```
<p>This text is <u>underlined</u></p>
```

It's important to note that these tags are just a way to add formatting to your text. They don't change the meaning of the text itself, and they don't necessarily affect how the text is read by a screen reader or other assistive technology. If you want to convey the meaning or importance of your text, it's often better to use semantic HTML tags like `<strong>` or `<em>` rather than visual tags like `<b>` or `<u>`.

Download the **`index.html`** file of [lesson 7](#) and open it with your browser. You will see the document built on lesson 6 with bold, italic, and underlined formatting.

1.14 Creating Hyperlinks in HTML

A link, or hyperlink, is a way to navigate between web pages on the internet. In HTML, a link is created using the `<a>` tag, which stands for “anchor”. The `<a>` tag is used to define the start and end of a link, and the `href` attribute is used to specify the destination of the link.

Here’s an example of a basic HTML link:

```
<a href="https://www.example.com">This is a link</a>
```

When this link is clicked, the user’s web browser will navigate to the URL specified in the `href` attribute (in this case, “https://www.example.com”). The text between the `<a>` and `</a>` tags (in this case, “This is a link”) is what the user will see and click on.

You can also use the `target` attribute to specify how the link should be opened. For example, you can use `target="_blank"` to open the link in a new tab or window.

```
<a href="https://www.example.com" target="_blank">This is a link that opens in a new  
↪ tab</a>
```

In addition to linking to other web pages, you can also use the `<a>` tag to create links within the same page. This is known as an “anchor” link, and it’s created by using the `id` attribute to give an element a unique name and then using that name as the destination of the link.

Here’s an example:

```
<h1 id="section1">This is a heading</h1>  
<p>Here's a link to <a href="#section1">the heading above</a></p>
```

When this link is clicked, the user’s web browser will jump to the element with the `id` of “section1” (in this case, the `<h1>` heading). This can be useful for creating a table of contents or for allowing users to quickly navigate to different sections of a page.

In [lesson 08](#) I added four new HTML files similar to `index.html`. Navigation through the five pages is accomplished by five links placed at the beginning of each file using code like this.

```
<a href="index.html">Click here to go to the first page</a><br/>  
<a href="second_page.html">Click here to go to the second page</a><br/>  
<a href="third_page.html">Click here to go to the third page</a><br/>  
<a href="fourth_page.html">Click here to go to the fourth page</a><br/>
```

```
<a href="fifth_page.html">Click here to go to the fifth page</a><br/>
```

1.15 Creating Lists with HTML: Ordered and Unordered Lists

HTML provides two types of lists for displaying content: ordered lists and unordered lists.

An ordered list is created using the `<ol>` element and is used to display a list of items in a specific order. Each item in the list is represented by a `<li>` element and is automatically numbered.

```
<ol>
  <li>First item</li>
  <li>Second item</li>
  <li>Third item</li>
</ol>
```

An unordered list is created using the `<ul>` element and is used to display a list of items that do not need to be in a specific order. Each item in the list is represented by a `<li>` element and is typically displayed with bullet points.

```
<ul>
  <li>First item</li>
  <li>Second item</li>
  <li>Third item</li>
</ul>
```

You can also nest lists within lists by using `<ol>` or `<ul>` elements inside the `<li>` elements.

```
<ul>
  <li>First item</li>
  <li>Second item
    <ul>
      <li>Sub-item 1</li>
      <li>Sub-item 2</li>
    </ul>
  </li>
  <li>Third item</li>
</ul>
```

It's worth noting that, like many other elements in HTML, lists are semantic elements, meaning

they provide information about the content they contain but do not affect the appearance of the content. You can use CSS to control the appearance of lists and list items, such as changing the bullet point or numbering style.

In [lesson 9](#) I used the unordered list to create a menu at the beginning of each page. This menu will contain the five links introduced in lesson 8. Here is the result.

In addition, I added an ordered list to each page as a demonstration. Here the result.

1.16 Enhancing Your Webpage with Images: The Tag

HTML provides a way to embed images into a webpage using the tag. The tag is a self-closing tag, meaning it does not have a closing tag, and is used to specify the source of the image, as well as various other attributes.

The two main attributes used when adding an image to an HTML document are the src and alt attributes.

The src attribute is used to specify the URL or file path of the image file. This attribute is required for the tag to function correctly.

The alt attribute is used to provide a text description of the image, which can be displayed if the image is not available or cannot be displayed for some reason. The alt attribute is also used by screen readers for accessibility purposes.

```

```

As with text, even for images you can use fake images to create your website. For example, you can use the [Lorem Picsum](#) website and with a URL like this <https://picsum.photos/200/300> you can create a 200x300 pixels image. If, on the other hand, you want a square image of 200 pixels just use the URL <https://picsum.photos/200>.

For our example website I created two to four images for each page and placed them in an img folder. In the pages, I have referenced these images with the tag as shown in the example above. The result can be seen below. The source code can be found in the [lesson-10](#) folder of our git project.

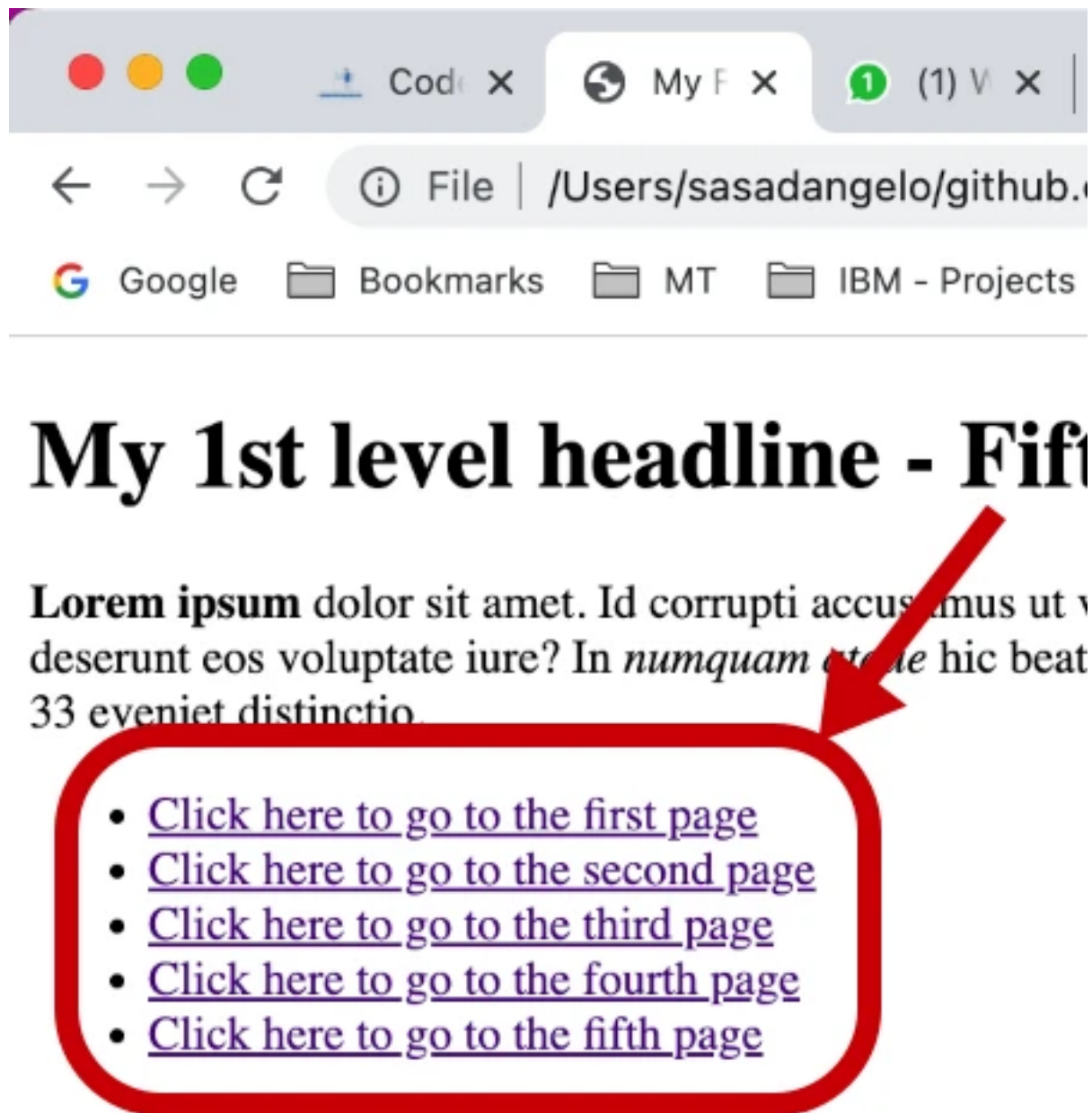


Figure 1.6: HTML Menu

My 3rd level headline - Fifth Page

Aut quia sint sit debitis voluptas eum culpa dolore est voluptate ea autem sunt cum nostrum quam sed voluntatem dolorem!

1. Et corporis veniam et delectus dolorem.
2. Est culpa nulla ea molestias enim non rerum iure!
3. Rem eveniet galisum vel nihil quae id iste exercitation

Figure 1.7: HTML Ordered List

1.17 Conclusion

In this article we covered:

- What HTML is and a brief history of its versions
- The basic structure of an HTML document: DOCTYPE, <html>, <head>, <body>
- Elements, tags, and attributes
- Headings (<h1>–<h6>) and paragraphs (<p>)
- Text formatting: bold, italic, underline
- Hyperlinks and anchor navigation
- Ordered and unordered lists
- Embedding images with

The [next article](#) covers CSS theory — selectors, the box model, typography, colours, and responsive design. After that, [Part 2 of the series](#) applies those concepts to build a real styled layout with a header, navigation bar, and responsive home page.

My 2nd level headline - First Page

Aut velit impedit eos **fugiat praesentium** qui reiciendis iste qui dolores et voluptas assumenda ut ducimus inventore. Ut modi adipisci id quidem necessitatibus ea doloremque quia ex *accusamus numquam*. Et quibusdam labore omnis et sapiente voluptatem ut doloribus galisum et assumenda : omnis.



Figure 1.8: HTML Images

Chapter 2

Mastering the Basics of CSS: A Beginner's Guide

2.1 Introduction

In the [previous article](#) you built a five-page HTML website from scratch. The pages work, but they have no style at all. This article covers the CSS concepts you need to change that — and ends by applying them to the project.

You should read this article if:

- you finished the HTML article and want to understand how styling works before applying it
- you know CSS properties exist but have never understood how selectors, cascading, or the box model actually interact
- you want a solid theory foundation before building a full layout in the [next part of the series](#)

2.2 What is CSS?

Photo from <https://www.scaler.com>

CSS (Cascading Style Sheets) is a styling language used to control the presentation of web pages. It is used to define the layout, colors, typography, and other visual elements of a web page. CSS allows you to separate the presentation of a web page from its structure, which is defined using HTML.

CSS has become an essential part of web development, as it allows you to create visually appeal-



Figure 2.1: CSS Logo

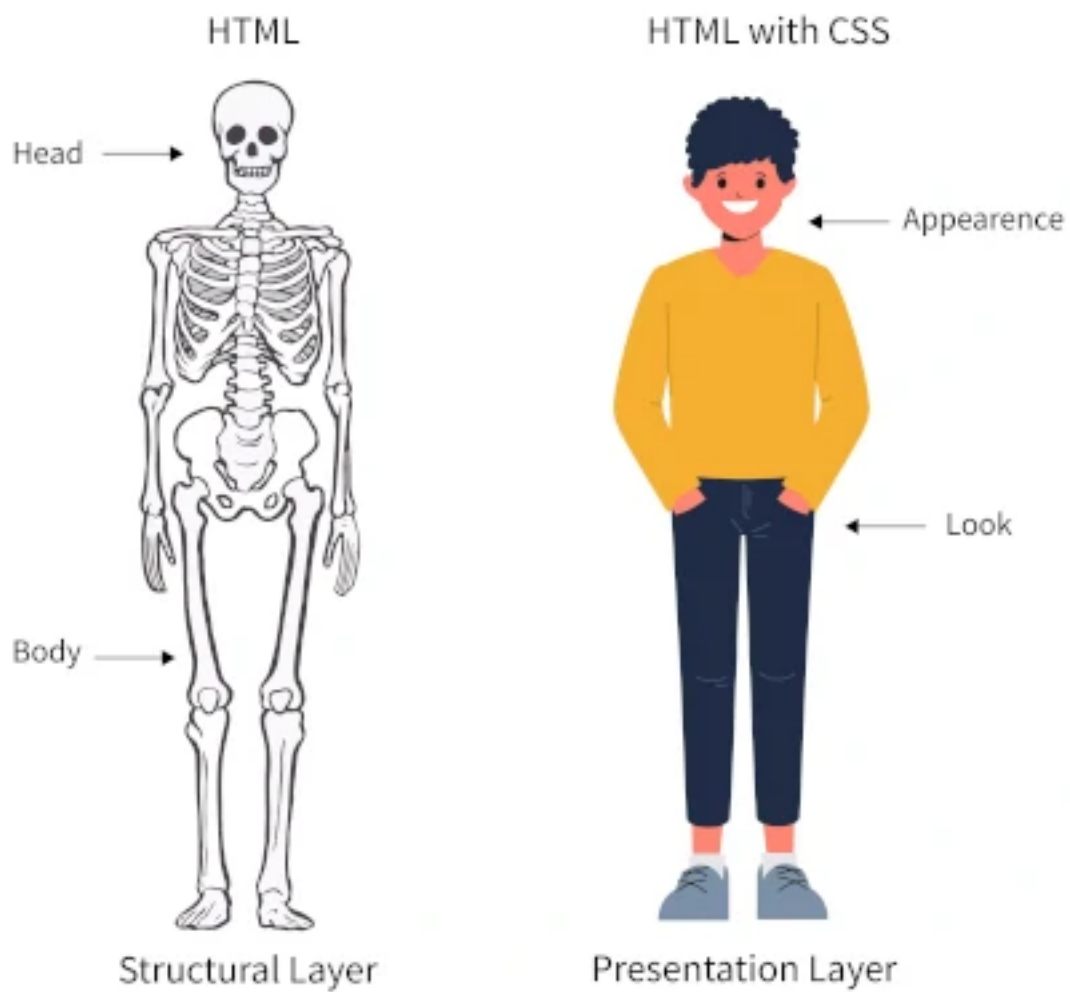


Figure 2.2: HTML + CSS



Figure 2.3: CSS Rules and Declarations

ing and responsive designs that adapt to different screen sizes and devices. With CSS, you can control the position and layout of elements, create animations and transitions, and define the typography and colors of a web page.

In this article, we will explore the basics of CSS and how it can be used to enhance the visual design of web pages. We will cover topics such as selectors, cascading, the box model, layout, typography, color and background, transitions and animations, and responsive design. By the end of this article, you will have a solid understanding of how to use CSS to create visually stunning and responsive web pages.

2.3 CSS Rules and Declarations

Photo from <https://www.coderepublics.com>

A CSS file is essentially a list of CSS rules, where each rule consists of a selector and a declaration block. A selector determines which HTML elements on a page will be affected by the styles defined in the rule. For example, an h1 selector would apply styles to all h1 headings on a page. A declaration block is a set of styles to be applied to the selected element. It consists of a property and a value, separated by a colon and surrounded by curly braces. For example:

```
h1 {  
  color: blue;  
  font-size: 36px;  
  text-align: center;  
}
```

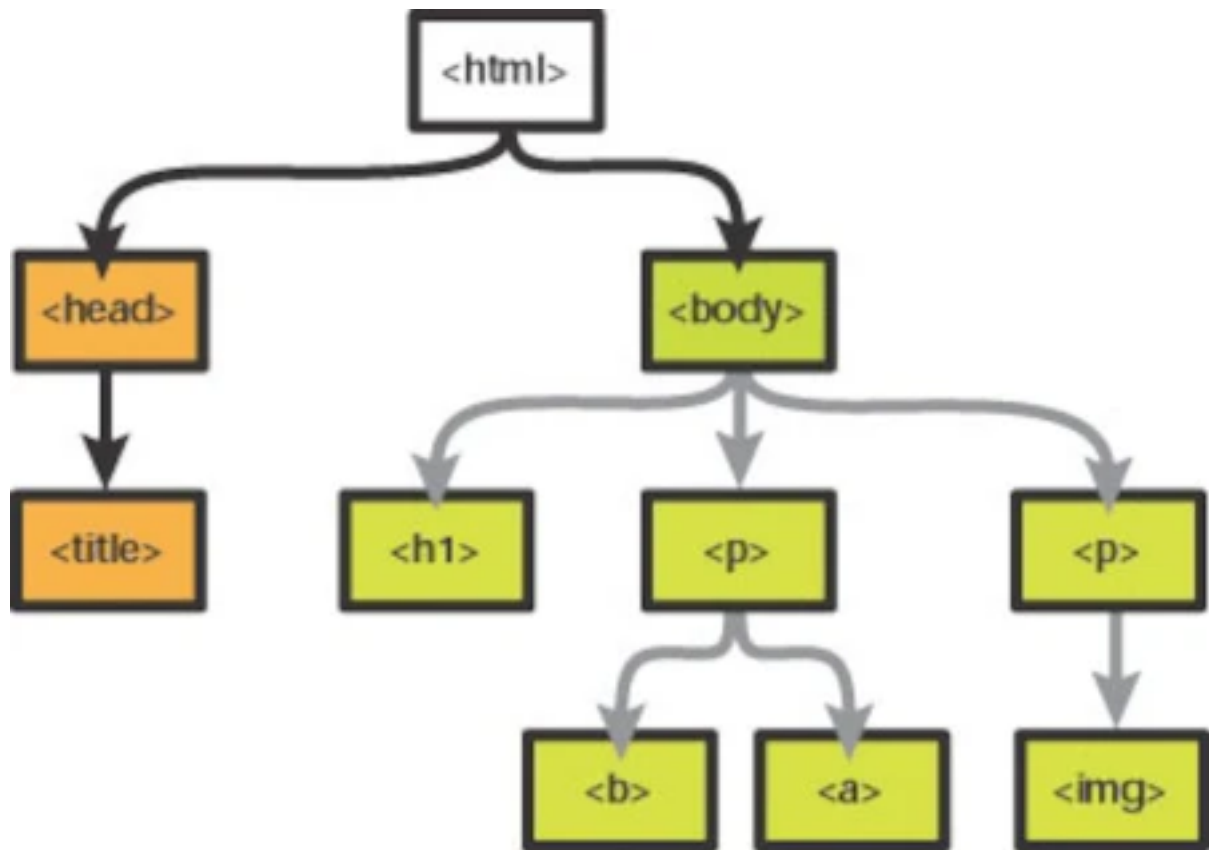


Figure 2.4: CSS Inheritance

In this example, the selector `h1` applies the styles to all `h1` headings on the page. The declaration block sets the text color to blue, the font size to 36 pixels, and the text alignment to the center. It's important to note that multiple CSS rules can apply to a single HTML element, and the order of the rules in the CSS file determines which styles will take priority (known as the cascading effect). The last style declared for a given property wins and overrides any previous styles.

2.4 CSS Inheritance

Photo from <https://sites.google.com/site/csstricksandtips>

In CSS, inheritance refers to the mechanism by which certain CSS property values are passed from a parent element to its children. This allows for a more efficient and organized way to apply styles to elements in an HTML document.

For example, if you set the `font-size` property on a `body` element, all its child elements will also inherit that `font-size` property value. If you then set the `font-size` property on a child element, it will override the inherited value and apply its unique `font-size` value.

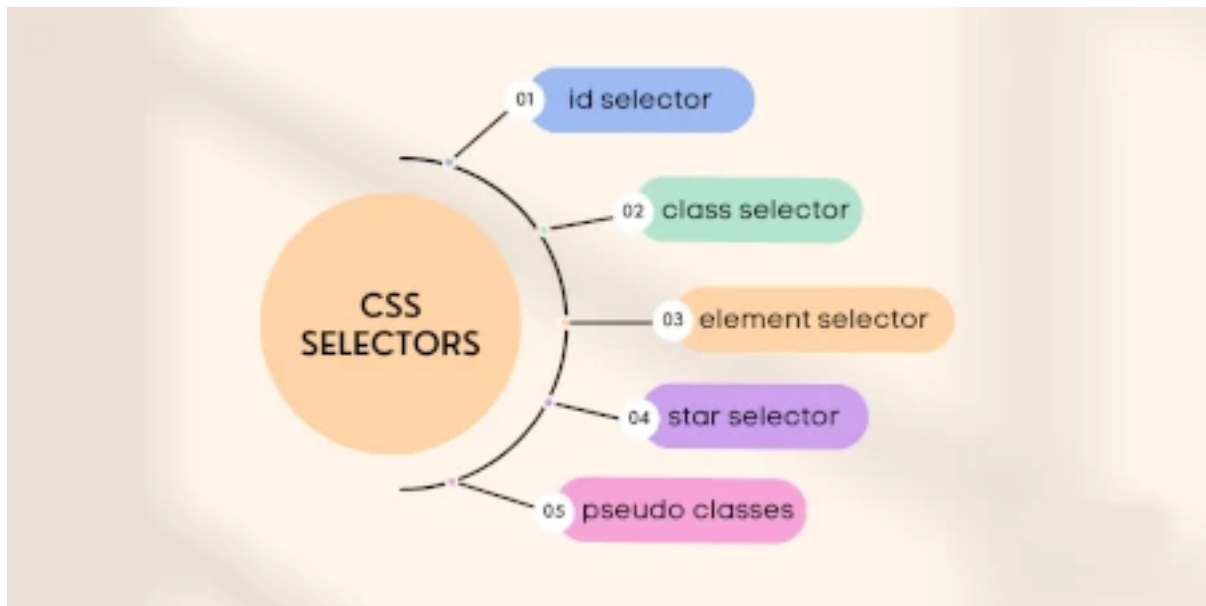


Figure 2.5: CSS Selectors

It's important to note that not all CSS properties are inherited. For example, the padding and margin properties are not inherited.

Inheritance can be a useful tool for maintaining consistency in styling and reducing the amount of code that needs to be written. However, it can also lead to unintended consequences if not used carefully. For example, if you have a large number of nested elements, inheritance can make it difficult to understand the styling of a particular element without carefully examining the hierarchy of elements in the HTML document.

2.5 CSS Selectors and Cascading: The Building Blocks of Web Design

Photo from <https://www.atatus.com>

In CSS, selectors are used to select elements on a web page that you want to apply styles. There are several types of selectors, including:

- **Element selectors:** Select elements based on their tag name, such as “p” for paragraphs or “a” for links.
- **Class selectors:** Select elements based on their class attributes, using a period (.) before the class name. For example, “.my-class” would select all elements with the class “my-class”.

- **ID selectors:** Select elements based on their id attribute, using a hash (#) before the id name. For example, “#my-id” would select the element with the id “my-id”.
- **Attribute selectors:** Select elements based on their attributes and attribute values. For example, “[href='https://example.com']” would select all elements with an href attribute equal to “https://example.com”.

Cascading refers to how styles are applied to elements when multiple stylesheets and/or styles are conflicting. When multiple styles apply to the same element, the browser uses the following rules to determine which style to use:

- **Specificity:** Styles with a higher specificity are given priority. For example, an ID selector has a higher specificity than a class selector.
- **Origin:** Styles that are defined in the user's browser have lower priority than those defined in the author's stylesheet.
- **Importance:** Styles that are marked as “!important” have higher priority than those that are not.

Finally, cascading also works with the order of styles in the CSS file, styles defined later on in the file will overwrite previous styles.

It's important to keep in mind specificity and cascading rules when working with CSS to avoid unexpected results.

Element Selector:

```
/* This will select all <p> elements and make their text color red */  
p {  
    color: red;  
}
```

Class selector:

```
/* This will select all elements with the class "my-class" and make their background  
↪ color yellow */  
.my-class {  
    background-color: yellow;  
}
```

ID Selector:

```
/* This will select the element with the ID "my-id" and make it's font-size 20px */  
#my-id {  
    font-size: 20px;  
}
```

Attribute Selector:

```
/* This will select all <a> elements with an href attribute equal to  
↪ "https://example.com" and make their text color blue */  
a[href='https://example.com'] {  
    color: blue;  
}
```

It's also possible to use attribute selectors to match elements with a specific attribute value prefix, suffix, or substring by using “^”, “\$”, “*” respectively.

```
/* This will select all <input> elements with a name attribute that starts with  
↪ "user" and make their border red */  
input[name^='user'] {  
    border: 1px solid red;  
}
```

It's important to note that attribute selectors are more performance-intensive than the other types of selectors, so it's best to use them sparingly and with caution.

Also, it's good practice to use class and ID selectors to select elements, instead of element selectors because it makes the CSS more reusable and less specific, avoiding the risk of changing the visual of unintended elements.

2.6 The Box Model

Photo from <https://www.geeksforgeeks.org/>

The box model is a concept in CSS that describes the layout of an HTML element as a rectangular box. Each box has four main parts: the content, padding, border, and margin.

- **Content:** This is the area where the element's content is displayed, such as text or images. A Content element has a size expressed by the properties' width and height. You can also use properties like min-width, min-height, max-width, and max-height to declare the

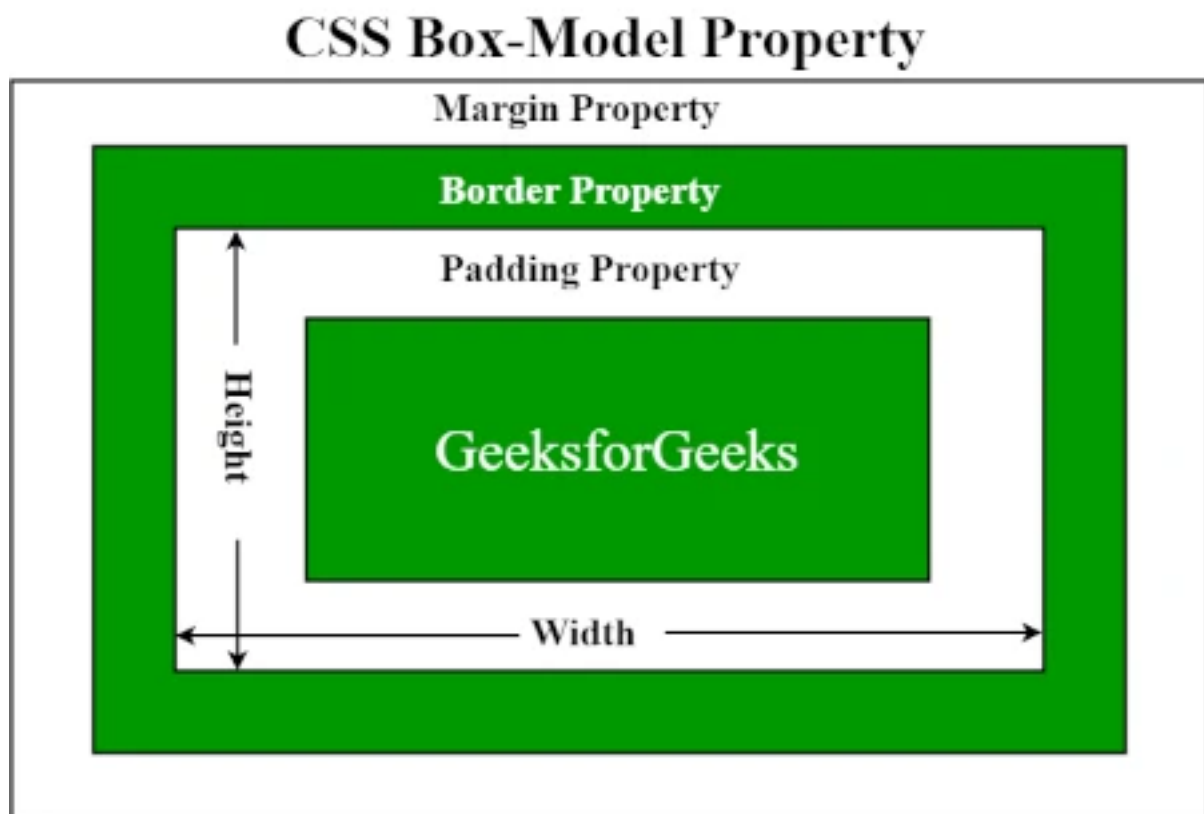


Figure 2.6: CSS Box Model

minimum and maximum size.

- **Padding:** This is the space between the content and the border. You can control the padding using the padding property and its related properties such as padding-top, padding-right, padding-bottom, padding-left.
- **Border:** This is the line that surrounds the padding and content. You can control the border using the border property and its related properties such as border-width, border-color, border-style.
- **Margin:** This is the space outside of the border. You can control the margin using the margin property and its related properties such as margin-top, margin-right, margin-bottom, margin-left.

When you set the width and height of an element, by default, it's the width and height of the content area. However, you can include the padding, border, and margin by using the box-sizing property with the value "border-box". The box-sizing property in CSS determines how the dimensions of an element are calculated. It specifies whether the width and height of an element should include the border and padding or not. There are two possible values for box-

sizing:

1. **content-box** (the default value): the dimensions of an element are determined only by its content, excluding the border and padding.
2. **border-box**: the dimensions of an element include both the content and the border and padding.

For example, if you specify a width of 200 pixels for an element with a 10-pixel border and 20-pixel padding, the element will have a total width of 250 pixels with `box-sizing: content-box` and 200 pixels with `box-sizing: border-box`. The `box-sizing` property can be useful for managing layout issues when working with fixed dimensions, as it allows for a more precise prediction of how spaces will be distributed.

```
/* This will make the width and height properties include the padding and border */  
.my-class {  
  box-sizing: border-box;  
  width: 200px;  
  height: 100px;  
  padding: 10px;  
  border: 1px solid #000;  
  margin: 20px;  
}
```

It's important to understand the box model and how the different parts interact with each other because it affects the layout and spacing of elements on a web page. Additionally, the `box-sizing` property can be very useful to control the dimensions of an element in a more intuitive way.

2.7 Layout in CSS

Photo from <https://www.reddit.com>

Layout in CSS refers to the positioning and arrangement of elements on a web page. There are several layout techniques that you can use to control the position of elements, such as:

- **display** property: This property is used to specify the type of box an element should generate. The most common values are `block`, `inline`, and `inline-block`. Block elements take up the full width of their parent container, and create a new line after them. Inline elements only take up as much width as their content and do not create a new line after

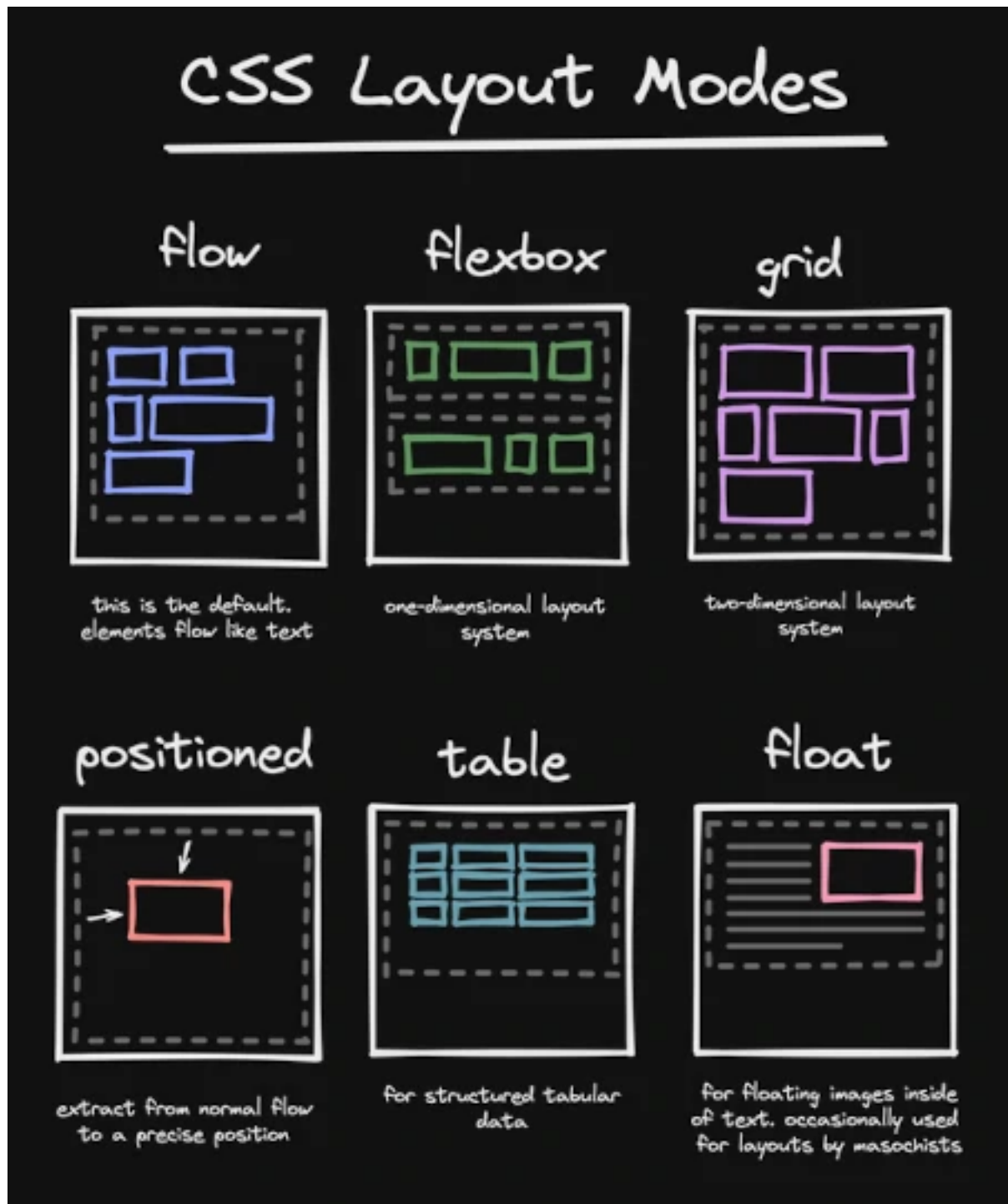


Figure 2.7: CSS Layouts

them. Inline-block elements are similar to inline elements, but they can have dimensions set with width and height properties, and they also create a new line after them.

- **position** property: This property is used to specify the positioning of an element. The possible values are static, relative, absolute, and fixed. Static is the default value and means that the element will be positioned according to the normal flow of the document. Relative means that the element will be positioned relative to its normal position, using the left, right, top, and bottom properties. Absolute means that the element will be positioned relative to its nearest positioned ancestor, using the same properties. Fixed means that the element will be positioned relative to the viewport, and it will remain in the same position even when the page is scrolled.
- **float** property: This property is used to make an element float to the left or right of its parent container. The possible values are left and right. When an element is floated, it will move to the left or right of its parent container, and other elements will flow around it.
- **flexbox** and **grid** layout: These are advanced layout techniques that are used to create flexible and responsive layouts. Flexbox is used to create a layout in one dimension, either a row or a column, and grid is used to create a layout in two dimensions, rows, and columns. Both flexbox and a grid provide powerful layout capabilities that allow you to control the size, alignment, and positioning of elements on a web page in a more flexibly and efficiently way.

It's important to keep in mind that the layout techniques should be used in combination to make the layout of a webpage more optimal and adaptable to different viewports. Additionally, it's good practice to use semantic HTML and to style according to the meaning of the elements, as this will make the layout more robust and maintainable.

2.8 Typography in CSS

Photo from <https://wpdaddy.com>

Typography in CSS refers to the control of the font, size, and spacing of text on a web page. Here are some key concepts in typography:

- **font-family**: This property is used to specify the font for an element. The value can be a specific font name, such as "Arial" or "Times New Roman", or a generic font family, such



Figure 2.8: CSS Typography

as “serif” or “sans-serif”. It’s also possible to specify a list of fonts, in case the first one is not available on the client device, the next one will be used.

- **font-size:** This property is used to specify the size of the text. The value can be specified in different units, such as pixels (px), points (pt), or ems (em).
- **font-weight:** This property is used to specify the boldness of the text. The possible values are normal and bold. It’s also possible to use numeric values from 100 to 900, where 400 is the same as normal and 700 is the same as bold.
- **line-height:** This property is used to specify the spacing between lines of text. The value can be specified in different units, such as pixels or ems.
- **letter-spacing** and **word-spacing:** These properties are used to specify the spacing between letters and words, respectively. The value can be specified in different units, such as pixels or ems.
- **text-align:** This property is used to specify the alignment of text within an element. The possible values are left, right, center, and justify.
- **text-transform:** This property is used to specify the capitalization of text. The possible values are none, uppercase, lowercase, and capitalize.
- **text-decoration:** This property is used to specify the decoration of text. The possible values are none, underline, overline, line-through, and blink (this one is not widely supported, use it with caution)

It’s important to keep in mind that typography is an important aspect of web design, and it can greatly impact the readability and aesthetics of a web page. Additionally, it’s good practice to use web-safe fonts and consider the accessibility guidelines when choosing the font family and size to ensure that the text is legible for users with visual impairments.

2.9 Color and Background in CSS

Photo from <https://edu.gcfglobal.org>

Color and background in CSS refer to the control of the color and background of elements on a web page. Here are some key concepts in color and background:

- **color:** This property is used to specify the text color of an element. The value can be



Figure 2.9: CSS Colors

specified in several ways, such as using color names (e.g. “red”), RGB values (e.g. “rgb(255, 0, 0)”), or hexadecimal values (e.g. “#ff0000”).

- **background-color:** This property is used to specify the background color of an element. The value can be specified in the same ways as the color property.
- **background-image:** This property is used to specify an image as the background of an element. The value is the URL of the image file.
- **background-repeat:** This property is used to specify how the background image should be repeated. The possible values are repeat, repeat-x, repeat-y, and no-repeat.
- **background-position:** This property is used to specify the position of the background image. The value can be specified in several ways, such as using keywords (e.g. “center”), length units (e.g. “10px 20px”), or percentages (e.g. “50% 50%”).
- **background-size:** This property is used to specify the size of the background image. The value can be specified in several ways, such as using keywords (e.g. “cover”), length units (e.g. “200px 100px”), or percentages (e.g. “50% 50%”).
- **background-clip** and **background-origin:** These properties are used to specify the area where the background image and color should be applied. The possible values of

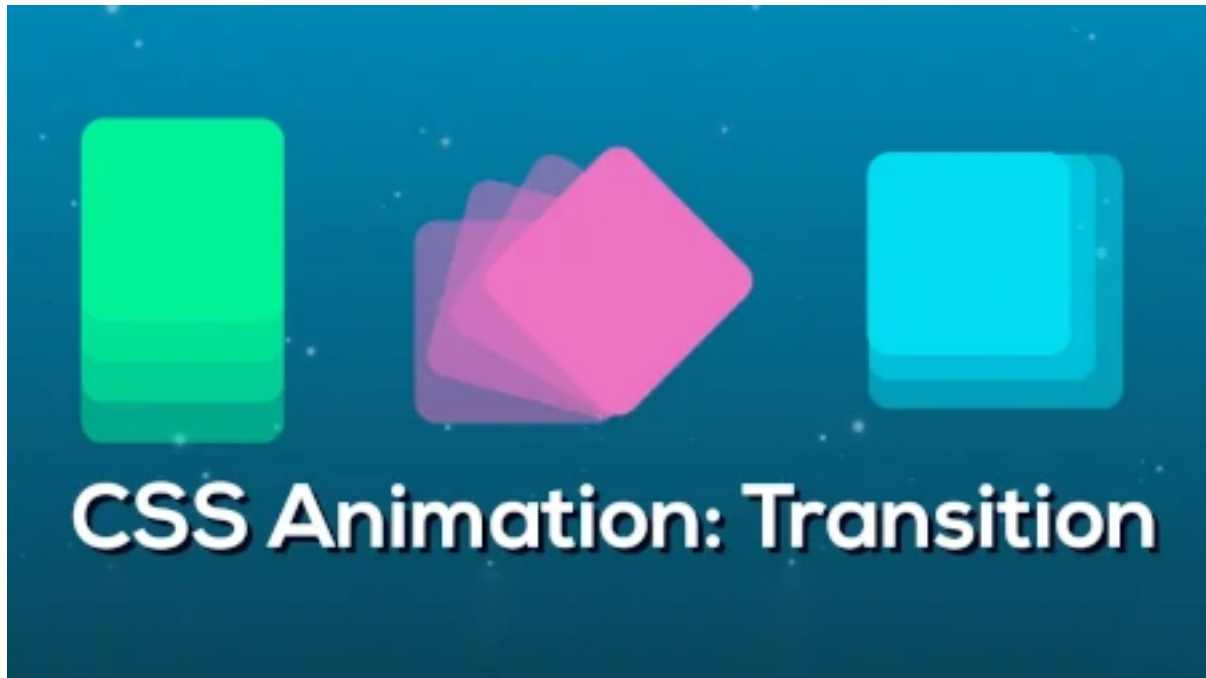


Figure 2.10: CSS Animations

background-clip are border-box, padding-box, and content-box, and the possible values of background-origin are padding-box and content-box.

It's important to keep in mind that color and background can greatly impact the aesthetics of a web page, and they should be used in a harmonious way to create a visually pleasing design. Also, it's good practice to consider the accessibility guidelines when choosing color combinations and contrasts to ensure that the text is legible for users with visual impairments.

2.10 Transitions and Animations in CSS

Photo from <https://edu.gefglobal.org>

Transitions and animations in CSS are used to create smooth and fluid visual effects when elements change their state.

- **transition:** This property is used to specify the transition effect that should be applied to an element when its state changes. The property takes several values, such as the property that will be transitioned (e.g. "color"), the duration of the transition (e.g. "1s"), the timing function (e.g. "ease-in-out"), and the delay before the transition starts (e.g. "0.5s").

```
/* This will make the transition of the color property to last 1 second, with an  
↪ ease-in-out timing function and a delay of 0.5s */  
.my-class {  
    transition: color 1s ease-in-out 0.5s;  
}
```

- **animation:** This property is used to specify a series of keyframes that define the animation effect. The property takes several values, such as the name of the animation (e.g. “my-animation”), the duration of the animation (e.g. “1s”), the timing function (e.g. “ease-in-out”), the number of times the animation should be played (e.g. “infinite”), and the delay before the animation starts (e.g. “0.5s”).

```
/* This will make the animation to be named 'my-animation', last 1 second, with an  
↪ ease-in-out timing function, to be played infinite times and a delay of 0.5s */  
.my-class {  
    animation: my-animation 1s ease-in-out infinite 0.5s;  
}
```

The keyframes need to be defined separately, usually in a different CSS file or in a style tag. The keyframes rule is used to define the styles that the animation should have at different points in time.

```
@keyframes my-animation {  
    0% {  
        transform: rotate(0deg);  
    }  
    100% {  
        transform: rotate(360deg);  
    }  
}
```

This animation will rotate the element from 0 degrees to 360 degrees in 1 second, with an ease-in-out timing function, and will repeat infinitely with a delay of 0.5s.

It's important to keep in mind that transitions and animations can greatly enhance the user experience and make a web page more interactive. However, it's good practice to use them in a subtly and appropriately way, to avoid overwhelming the user and to make sure that the animation does not interfere with the functionality of the page.



Figure 2.11: CSS Responsive Design

2.11 Responsive Design in CSS

Photo from <https://www.tutorialspoint.com>

Responsive design in CSS is used to create web pages that adapt to different screen sizes and resolutions. The main idea behind responsive design is to use flexible grids, flexible images, and media queries to create a layout that adjusts to the size of the viewport.

- **media queries:** This is a CSS technique that allows you to apply different styles based on the characteristics of the device and the viewport. Media queries use a set of CSS rules that are only applied when certain conditions are met, such as the width or height of the viewport, the resolution of the screen, or the type of device.

```
/* This media query will apply the styles inside of it when the width of the viewport  
↪ is less than 600px */  
@media only screen and (max-width: 600px) {  
    /* styles go here */  
}
```

- **flexbox** and **grid layout:** These layout techniques are very useful for responsive design because they allow you to create flexible and adaptive layouts that adjust to the size of the



Figure 2.12: How to Include CSS in web page

viewport. Flexbox is used to create a layout in one dimension, either a row or a column, and a grid is used to create a layout in two dimensions, rows, and columns.

- **viewport:** This is the area of the browser window where the web page is displayed. The viewport can be controlled using the viewport meta tag, this will allow you to set the width and initial scale of the viewport, and it will affect the layout of the page and the sizes of elements on the page.

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

It's important to keep in mind that responsive design is crucial for creating a good user experience in today's web, where people access the internet from a wide range of devices and screen sizes. Additionally, it's good practice to test the layout on different devices and screen sizes during the development process to ensure that the layout is consistent and adaptable across different platforms.

2.12 How to Include CSS in a web page

Photo from <https://www.hostinger.com>

CSS can be included in a web page in three different ways: inline, internal, and external.

Inline CSS refers to writing CSS styles directly within an HTML element using the “style” attribute. This method is useful for applying styles to a specific element but is not recommended for larger stylesheets as it makes the HTML code cluttered. For example:

```
<p style="color: blue; font-size: 16px;">This is a paragraph with inline CSS</p>
```

Internal CSS refers to writing CSS styles within the head section of an HTML document, within a “style” tag. This method is better than inline CSS as it keeps the CSS styles separated from the HTML content but is still not recommended for larger stylesheets as it requires the same CSS code to be duplicated in each HTML file. For example:

```
<head>
  <style>
    p {
      color: blue;
      font-size: 16px;
    }
  </style>
</head>
```

External CSS refers to writing CSS styles in a separate file with a .css extension and linking it to an HTML document. This method is the best way to implement CSS in a web page as it separates the styles from the HTML content and allows for a single stylesheet to be used across multiple pages. This reduces the amount of code duplication and makes it easier to maintain the styles. To include an external CSS file, the “link” tag is used within the head section of the HTML document. For example:

```
<head>
  <link rel="stylesheet" type="text/css" href="style.css">
</head>
```

In conclusion, while inline and internal CSS are quick and easy to implement, it is best practice to use external CSS in web development for the maintainability and organization of code.

2.13 Applying CSS Principles to Create a Website Style

Photo from <https://fronty.com>

In order to enhance the look and feel of the 5 pages created in the [previous article](#), we will now create a CSS stylesheet that will apply the principles discussed earlier to these pages.

First, it is necessary to include the following line of code in each of the 5 HTML pages:



Figure 2.13: Applying CSS Principles

```
<link rel="stylesheet" type="text/css" href="style.css">
```

This line of code associates the HTML file with the new CSS file “style.css”.

Next, in the CSS file, we will define a basic layout for the page by applying the following code to the “body” element:

```
body {  
    margin: 0 auto;  
    font-family: 'Helvetica Neue';  
    font-size: 13px;  
    max-width: 1080px;  
}
```

This code tells the browser to set the top and bottom margins to 0, and to calculate the left and right margins automatically based on the page size and the size of the browser screen. The maximum width of the text is set to 1080px. Additionally, the code sets the default font size to 13px and the font type to ‘Helvetica Neue’.

For the headings (h1, h2, h3, h4, h5, h6), we set the following styles:

```
h1, h2, h3, h4, h5, h6 {  
    color: #3b3b3b;  
    text-align: left;  
}  
  
h1 {  
    font-size: 28px;  
}  
  
h2 {  
    font-size: 24px;  
}  
  
h3 {  
    font-size: 20px;  
}  
  
h4 {  
    font-size: 18px;  
}  
  
h5 {  
    font-size: 16px;  
}  
  
h6 {  
    font-size: 14px;  
}
```

The heading font sizes range from 14px for h6 to 28px for h1, and all headings will have a color of #3b3b3b and be left-aligned.

For the paragraphs, we set the text alignment to justify:

```
p {  
    text-align: justify;  
}
```

For ordered lists, we set the list style position to be inside the text:

```
ol {  
    list-style-position: inside;  
}
```

For blockquotes, we set the following styles:

```
blockquote {  
    background: #f5f7f9;  
    border-bottom: 1px solid #d8dee9;  
    padding: 10px 10px 10px 10px;  
}
```

The background color is set to #f5f7f9, the bottom border is set to 1px solid and colored #d8dee9, and there is a padding of 10px around the entire quote.

For images, we want them to alternate between appearing on the left and right of the text, with the text flowing around them. To achieve this, we will apply the following code:

```
img.img-left {  
    padding: 10px 10px 10px 0px;  
    float: left  
}  
  
img.img-right {  
    padding: 10px 0px 10px 10px;  
    float: right  
}
```

As you can see, the images have a padding of 10px all around except on the side where they are aligned. The float attribute expresses how the text should flow.

The entire code can be found in the [lesson-11 folder](#) of the repository.

2.14 Conclusion

In this article we covered:

- CSS rules, declarations, selectors, and the cascading mechanism
- Inheritance — which properties propagate and which do not

- The box model: content, padding, border, margin, and box-sizing
- Layout techniques: display, position, float, Flexbox, and Grid
- Typography: font-family, font-size, font-weight, line-height, text-align
- Colour and background properties
- Transitions and animations with @keyframes
- Responsive design with the viewport meta tag and media queries
- Three ways to include CSS: inline, internal, and external

The [next article](#) puts all of this into practice — building a real site layout with a header, navigation bar, social icons, a home page with a hero image, and a fully responsive mobile menu.

Chapter 3

Building a Professional Website Layout with CSS: Header, Navigation, and Responsive Design

3.1 Introduction

In the [previous articles](#) of this series you built a five-page HTML website from scratch, and in [Mastering the Basics of CSS](#) you learned the theory behind stylesheets. You have working pages, but they still look like a plain document from 1999: no header, no navigation bar, no branding, and nothing that adapts to a phone screen.

This article changes that. Following the [Part 2 of the html-hero project](#) step by step, I will show you how to turn those raw HTML pages into a visually consistent site with a real layout, social icons, a styled home page, and a responsive mobile menu.

You should read this article if:

- you completed the first HTML article and want to continue building the project
- you know basic CSS properties but have never applied them to a real multi-page layout
- you want to understand how header, footer, navigation bar, and responsive design actually work in practice — not just in theory

All the source code is available in the [part-2 folder of the html-hero repository](#). Each lesson has its own subfolder (lesson-13 through lesson-19) so you can compare the result at every step.

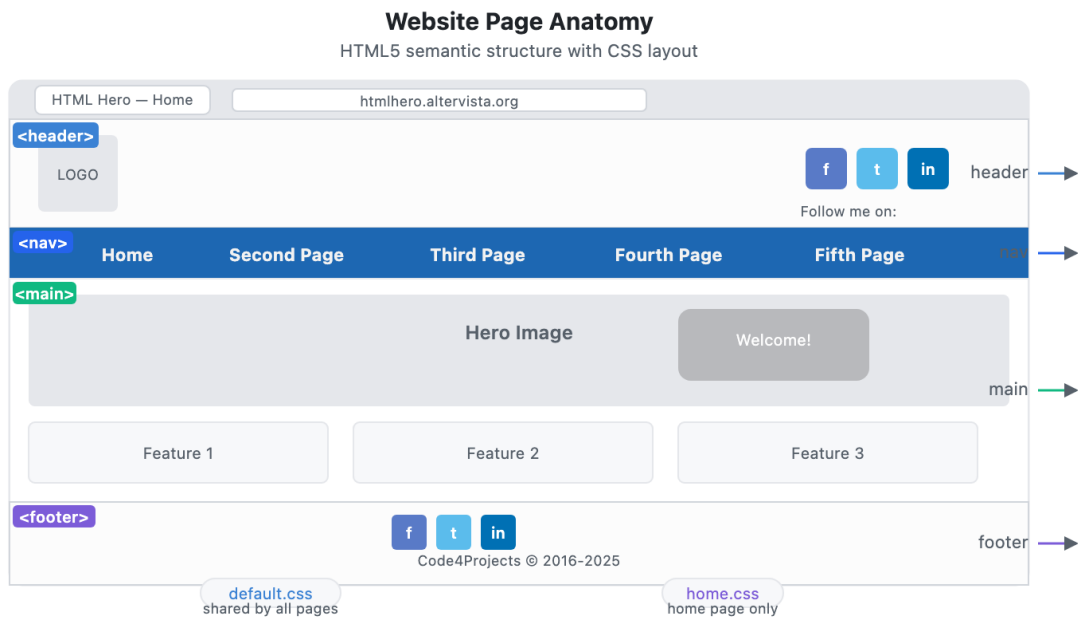


Figure 3.1: Building a Professional Website Layout with CSS

3.2 Your First CSS File

Before building the layout, let's quickly cover what Lesson 11 introduced at the end of Part 1: the first external CSS file.

Up to Lesson 10, all pages were unstyled HTML. Lesson 11 adds a `default.css` file linked from every page with a single `<link>` tag in the `<head>`:

```
<head>
  <meta charset="UTF-8">
  <title>HTML Hero</title>
  <link rel="stylesheet" type="text/css" href="assets/css/default.css">
</head>
```

This one stylesheet applies to all pages, which is exactly the right approach for a consistent site. Its base rules set the font, constrain the content width, and add basic typography:

```
body {
  margin: 0 auto;
  font-family: 'Helvetica Neue';
  font-size: 13px;
}
```

```
main {  
    max-width: 1080px;  
    margin-left: auto;  
    margin-right: auto;  
}
```

The `max-width` on `main` keeps the content readable on wide monitors while the `auto` margins keep it centred. This pattern is used throughout the entire project.

3.3 Publishing Your Website

Lesson 12 is about going live. The project uses [Altrivista](#), a free Italian hosting service, and [FileZilla](#) as the FTP client to upload files.

The steps are straightforward: create a free space on Altrivista, switch it from the default WordPress plan to a plain hosting plan, then connect FileZilla with the FTP credentials Altrivista provides, and upload all your files. This is optional — you can also follow along locally — but it is a useful exercise for understanding how websites actually get published.

3.4 Header and Footer with Semantic HTML

Part 2 starts properly here. The first thing every real website needs is a consistent header and footer on every page.

HTML5 introduced the `<header>` and `<footer>` tags specifically for these regions. Using them instead of a generic `<div>` has two benefits: the code is more readable, and search engines and screen readers understand the structure better.

```
<body>  
  <div id="page">  
    <header>  
      <div id="header-group">  
        <div id="logo">  
            
        </div>  
      </div>  
    </header>
```

```
<main>
  <!-- page content here -->
</main>
<footer>
  <div id="site-info">
    Code4Projects, Copyright Salvatore D'Angelo &copy; 2016–2025.
  </div>
</footer>
</div>
</body>
```

The CSS gives the header a light background and fixes its height. The `#header-group` container uses Flexbox to position the logo and, in later lessons, the social icons side by side:

```
header {
  background-color: #FCFCFC;
  width: 100%;
}

#header-group {
  padding: 0px 35px;
  max-width: 1080px;
  margin-left: auto;
  margin-right: auto;
  display: flex;
  flex-direction: row;
  justify-content: space-between;
}
```

The key takeaway: semantic tags (`<header>`, `<footer>`, `<nav>`, `<main>`) are not just cosmetic — they communicate the structure of your page to browsers, search engines, and assistive technologies.

This same header and footer markup is copied to all five HTML files. That repetition is deliberate at this stage: in a real project you would use a template engine or a framework to avoid it, but at the beginner level it is important to understand exactly what is being shared before abstracting it away.

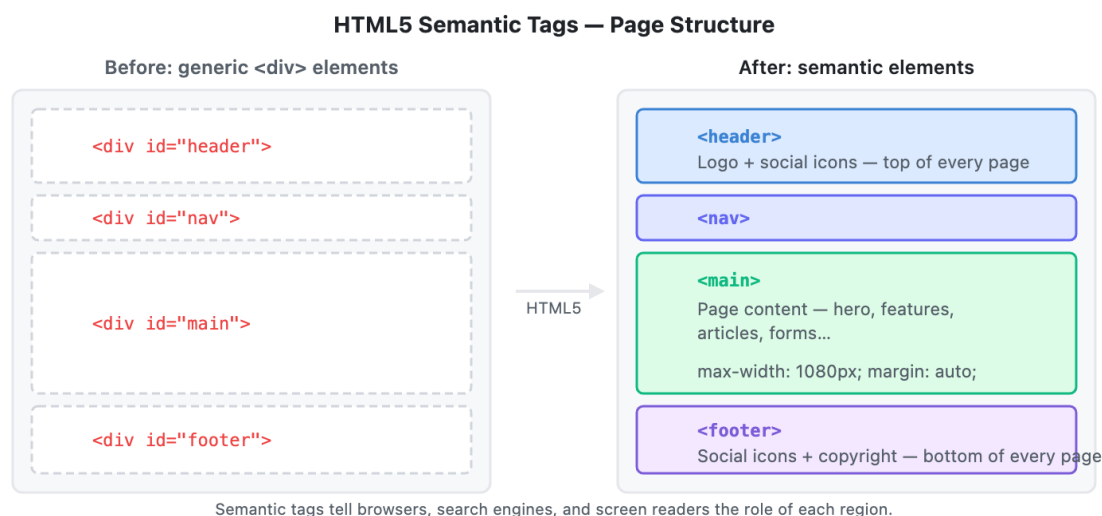


Figure 3.2: HTML5 Semantic Tags — Page Structure

3.5 Social Media Icons with Font Awesome

Plain text links for social profiles look dated. The industry standard is to use icon fonts, and [Font Awesome](#) is the most widely adopted one.

Font Awesome delivers icons as a web font, which means no image files are needed — each icon is a Unicode character rendered by a font file. The project uses a local copy of Font Awesome to avoid any external dependency. Two CSS files are added to the project:

- `assets/css/fontawesome.css` — core Font Awesome rules
- `assets/css/brands.css` — brand icon definitions (Facebook, Twitter, etc.)

These are linked from every page alongside `default.css`:

```
<link rel="stylesheet" type="text/css" href="assets/css/default.css">
<link rel="stylesheet" type="text/css" href="assets/css/brands.css">
<link rel="stylesheet" type="text/css" href="assets/css/fontawesome.css">
```

Adding a social icon button to the header is then a matter of one `<a>` tag with the right Font Awesome class:

```
<a aria-label="Code4Projects Facebook page"
  class="social-media-icon social-media-icon-facebook"
  href="https://www.facebook.com/code4projects/">
  <span class="fa-brands fa-facebook-f"></span>
```

```
</a>
```

The `social-media-icon` class controls the button size and shape (36×36 px, rounded corners). The `social-media-icon-facebook` class gives it the characteristic blue gradient background. The `fa-brands fa-facebook-f` classes on the `<span>` render the actual icon from the font file.

3.6 Completing the Footer

With Font Awesome already integrated, completing the footer is straightforward: reuse the same social icon markup inside `<footer>` and add the copyright text next to it.

```
<footer>
  <div id="footer-social-media-group">
    <div class="social-media-title">Follow me on:</div>
    <div class="social-media-icons">
      <!-- same icon markup as in the header -->
    </div>
  </div>
  <div id="site-info">
    Code4Projects, Copyright Salvatore D'Angelo &copy; 2016–2025.
  </div>
</footer>
```

The CSS uses Flexbox to stack the social area above the copyright text and keep everything centred. The important point here is **code reuse**: once you understand a pattern (icon button with Font Awesome), you apply it in multiple places without reinventing it.

3.7 The Navigation Bar

Every multi-page site needs a way to move between pages. A navigation bar is the standard solution, and HTML already has a semantic element for it: `<nav>`.

The markup uses an unordered list inside `<nav>`. At this point the links still point to the five original pages by filename:

```
<div id="navigation">
  <nav>
    <ul>
```

```
<li><a href="index.html">Home</a></li>
<li><a href="second_page.html">Second Page</a></li>
<li><a href="third_page.html">Third Page</a></li>
<li><a href="fourth_page.html">Fourth Page</a></li>
<li><a href="fifth_page.html">Fifth Page</a></li>
</ul>
</nav>
</div>
```

By default an unordered list is vertical. CSS transforms it into a horizontal blue bar:

```
#navigation {
    background-color: #1D67B2;
}

nav {
    padding: 5px;
    max-width: 1080px;
    margin-left: auto;
    margin-right: auto;
}

nav ul li {
    padding: 11px 35px;
    display: inline-block;
}

nav ul li a {
    color: #f2f2f2;
    text-decoration: none;
    font-weight: 700;
}
```

`display: inline-block` is the key rule: it tells each `<li>` to sit side by side instead of stacking. The `#navigation` div gets the blue background, and the link colour is set to near-white so it is readable against that background.

3.8 Building the Home Page

Inner pages (the five content pages) all share the same basic structure. The home page is different: it needs a visual impact — a hero image, sections with icons, and featured content. This is why a second CSS file, `home.css`, is introduced and loaded only by `index.html`.

3.8.1 The Hero Area

The hero area fills the full width with a background image and overlays a text box on top of it:

```
.hero-background {
    background-image: url("../assets/img/code4projects-image-hero.webp");
    background-position: center;
    background-repeat: no-repeat;
    background-size: cover;
    position: relative;
}

.hero-box {
    padding: 50px 40px;
    display: flex;
    flex-direction: row;
    justify-content: flex-end;
}

.hero-messagebox {
    padding: 50px 40px;
    max-width: 400px;
    background-color: RGBA(139, 139, 139, 0.5);
    border-radius: 20px;
}
```

`background-size: cover` scales the image to fill the entire element without distortion. `justify-content: flex-end` pushes the text box to the right side. The semi-transparent grey background (RGBA with 0.5 alpha) makes the text legible over any background image.

3.8.2 The Newsletter Section

A newsletter sign-up section sits below the feature rows. It uses Flexbox to place an eBook cover image and the sign-up form side by side:

```
<div class="newsletter-home-box">
  <div class="newsletter-home-image">
    
  </div>
  <div class="newsletter-home-form">
    <div class="newsletter-home-text-title">Download Free eBook</div>
    <form action="#" method="get" target="_blank">
      <input class="newsletter-home-input" name="name"
        type="text" placeholder="Name" required>
      <input class="newsletter-home-input" name="email"
        type="email" placeholder="Email" required>
      <button type="submit" class="primary">Download</button>
    </form>
  </div>
</div>
```

The `type="email"` attribute on the email field gives you basic format validation for free — the browser will reject submissions that do not look like an email address, without any JavaScript needed.

3.9 Making the Site Responsive

A site that works on desktop but breaks on a phone is not a finished site. The final step of Part 2 adds responsive design, which means the layout adapts automatically to the screen size.

3.9.1 The Viewport Meta Tag

The first change is the simplest: add this `<meta>` tag to the `<head>` of every page.

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Without it, mobile browsers scale the page down to fit, rendering it as a shrunken desktop version. With it, the browser uses the actual device width and the layout rules you define via media queries.

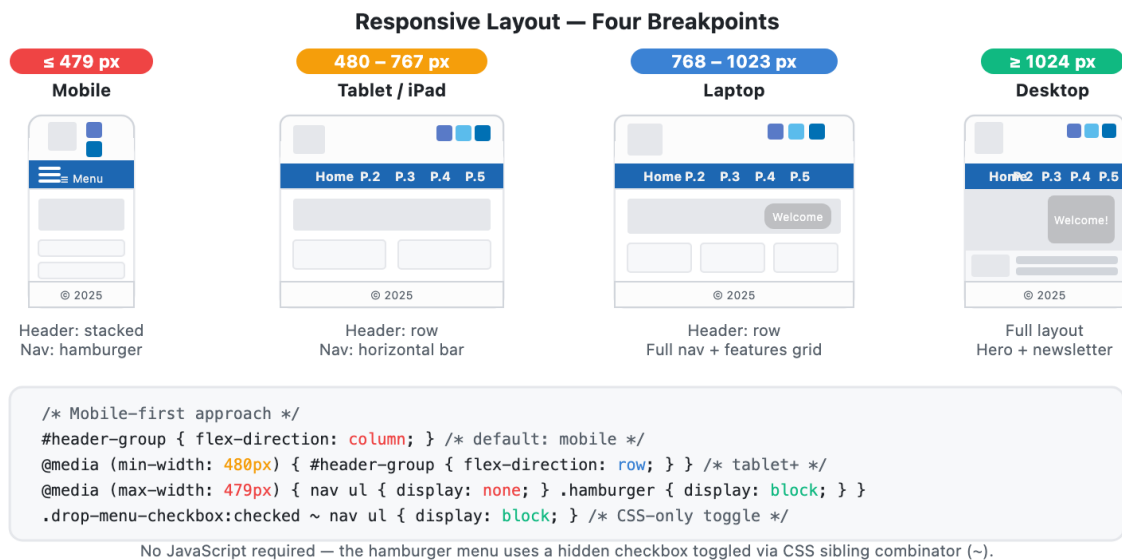


Figure 3.3: Responsive Layout — Four Breakpoints

3.9.2 Media Queries

The project targets four breakpoints. The diagram below shows how the layout changes at each one:

The project targets four breakpoints:

Name	Width range
Mobile phones	≤ 479 px
Tablets / iPad	480 px – 767 px
Laptop / small screen	768 px – 1023 px
Desktop	≥ 1024 px

The CSS follows a **mobile-first** approach: the default rules are written for mobile, then overridden for wider screens using `@media` screen and `(min-width: ...)`. For example, the header layout is vertical by default and switches to horizontal row on tablets and above:

```

/* Default: mobile – vertical stack */
#header-group {
  display: flex;
  flex-direction: column;
  justify-content: center;
}

```

```
/* Tablets and wider: horizontal row */
@media screen and (min-width: 480px) {
    #header-group {
        flex-direction: row;
        justify-content: space-between;
    }
}
```

3.9.3 The Hamburger Menu

On a phone, a horizontal navigation bar does not fit. The solution is a hamburger menu: three horizontal lines that, when tapped, reveal the navigation links in a dropdown.

The implementation uses a hidden checkbox as a CSS-only toggle — no JavaScript required:

```
<div id="navigation">
  <input class="drop-menu-checkbox" type="checkbox" id="drop-menu-checkbox">
  <label class="hamburger" for="drop-menu-checkbox">
    <span class="hamburger-line"></span>
  </label>
  <nav>
    <ul>
      <li><a href="index.html">Home</a></li>
      <!-- ... -->
    </ul>
  </nav>
</div>
```

The three hamburger lines are drawn entirely in CSS using `::before` and `::after` pseudo-elements on `.hamburger-line`. When the checkbox is checked, a CSS rule makes the `<ul>` visible:

```
/* Mobile: nav list hidden by default */
@media screen and (max-width: 479px) {
    nav ul {
        display: none;
    }
}
```

```
/* Show nav when checkbox is checked */
.drop-menu-checkbox:checked ~ nav ul {
    display: block;
}

/* Hide hamburger on screens wider than mobile */
@media screen and (min-width: 480px) {
    .hamburger {
        display: none;
    }
}
```

The `~` is the CSS **general sibling combinator**: it selects `nav ul` when `.drop-menu-checkbox` (the hidden checkbox) is a preceding sibling. Clicking the hamburger label toggles the checkbox, which toggles the menu — no JavaScript at all.

3.10 The Final File Structure

After all Part 2 lessons, the project structure looks like this:

```
lesson-19/
├─ index.html
├─ second_page.html
├─ third_page.html
├─ fourth_page.html
├─ fifth_page.html
├─ favicon.ico
├─ assets/
│   └─ css/
│       ├── default.css      ← shared by all pages
│       ├── home.css         ← home page only
│       ├── fontawesome.css  ← Font Awesome core
│       └─ brands.css        ← Font Awesome brand icons
│   └─ fontawesome/
│       └─ webfonts/         ← .ttf and .woff2 font files
└─ img/                     ← site images
```

The separation between `default.css` (shared rules) and `home.css` (home-page-only rules) is the key architectural decision. It keeps the shared stylesheet focused and makes it clear what is

specific to the home page.

3.11 Conclusion

In this article we covered:

- How to use `<header>` and `<footer>` semantic tags for a consistent page structure
- How to integrate Font Awesome locally to add scalable social media icons
- How to build a horizontal navigation bar from a `<ul>` with `display: inline-block`
- How to structure a home page with a CSS background image hero, circular icons, and feature sections
- How to add a newsletter sign-up form with styled `<input>` fields
- How to apply responsive design with the viewport meta tag, media queries, and a CSS-only hamburger menu

The [next article](#) builds out the real content pages: About Me, Start Here, Resources, a Contacts form, and an interactive photo gallery — introducing HTML forms, CSS Grid, and the first JavaScript in the series.

Chapter 4

Building the Complete Website: Real Pages, HTML Forms, and a Photo Gallery

4.1 Introduction

In the [previous article](#) you gave your site a real shell: a consistent header and footer, a navigation bar, social media icons, a styled home page, and a responsive hamburger menu. The five inner pages still had placeholder content — just a heading and a paragraph.

This is Part 4 of the series. If you missed the earlier articles, you can start from [Getting Started with HTML](#), [Mastering the Basics of CSS](#), or [Building a Professional Website Layout with CSS](#).

This article builds out those pages for real. Following the [Part 3 of the html-hero project](#) lesson by lesson, I will show you how to write the About Me, Start Here, and Resources pages, how to build a contact form with HTML form elements, and how to create an interactive photo gallery using CSS Grid and a handful of lines of JavaScript.

You should read this article if:

- you completed Part 3 and want to move beyond layout into real content pages
- you have never built an HTML form and want to understand how `<input>`, `<label>`, `<textarea>`, and `<button>` work together
- you want to understand CSS Grid and how it differs from Flexbox for image-based layouts
- you want to add your first JavaScript to a web page without reaching for a framework

All the source code is available in the [part-3 folder of the html-hero repository](#). Each lesson has its own subfolder (lesson-20 through lesson-24) so you can compare the result at every step.



Figure 4.1: Building the Complete Website: Real Pages, HTML Forms, and a Photo Gallery

4.2 A New CSS File: page.css

Part 2 used two CSS files: `default.css` (shared by every page) and `home.css` (home page only). Part 3 introduces a third file, `page.css`, loaded by all inner pages.

The separation is deliberate. The home page has a hero image, feature rows, and a newsletter section — styles that make no sense on an article page. Inner pages share a different set of needs: a centred content container, floating images, styled blockquotes, form fields, and gallery cells.

```
<link rel="stylesheet" type="text/css" href="assets/css/default.css">
<link rel="stylesheet" type="text/css" href="assets/css/page.css">
<link rel="stylesheet" type="text/css" href="assets/css/brands.css">
<link rel="stylesheet" type="text/css" href="assets/css/fontawesome.css">
```

The base rules in `page.css` centre the content area and handle text and image layout:

```
#page-container {
  max-width: 1080px;
  margin-left: auto;
  margin-right: auto;
  padding: 10px;
  overflow: hidden;
}
```

```
p {
  color: #666666;
  text-align: justify;
}

img.img-left {
  float: left;
  padding: 10px 10px 10px 0px;
}

img.img-right {
  float: right;
  padding: 10px 0px 10px 10px;
}
```

The `overflow: hidden` on `#page-container` is a classic float-clearing trick: it forces the container to expand around any floated images inside it, preventing them from overflowing outside the box.

4.3 The About Me Page

The About Me page is the simplest of the five — it follows the same article-style structure as any inner page: a heading, paragraphs of text, and images floated left or right to break the visual monotony.

```
<main class="page-main">
  <div id="page-container">
    <h1>About Me</h1>
    
    <p>
      My name is Salvatore D'Angelo and I am a professional software engineer
    </p>
  </div>
</main>
```

The `img-left` class floats the profile picture to the left so the text wraps around it on wider

screens. On mobile (below 480 px) the float is removed and the image sits above the paragraph, which reads more naturally on a narrow screen:

```
/* Mobile: no float, image sits above text */
img.img-left {
  padding: 10px;
}

/* Tablets and wider: float left */
@media screen and (min-width: 480px) {
  img.img-left {
    float: left;
    padding: 10px 10px 10px 0px;
  }
}
```

The float property was the original CSS layout mechanism before Flexbox and Grid existed. It is still the right tool when you want text to wrap around an image.

4.4 The Start Here Page and <figure>

The Start Here page introduces two semantic HTML elements you have not seen yet: <figure> and <figcaption>.

A <figure> groups an image with its caption as a single semantic unit. Search engines and screen readers understand that the caption belongs to the image — something a plain followed by a <p> does not communicate:

```
<figure>
  
  <figcaption>
    Photo from
    <a href="https://clipartstation.com/">clipartstation.com</a>
  </figcaption>
</figure>
```

The CSS in page.css handles the caption and link styling:

```
figure {  
    margin: 0px;  
}  
  
figure figcaption a {  
    color: #1D67B2;  
}
```

The `margin: 0` overrides the browser default margin on `<figure>` (some browsers add 40 px of horizontal margin by default), which would otherwise push the figure away from the page edges unexpectedly.

4.5 The Resources Page

The Resources page is a curated list of links. It uses `<ol>` for ordered lists and demonstrates the `target="_blank"` attribute, which opens links in a new tab:

```
<a href="https://developer.mozilla.org/en-US/"  
    target="_blank" rel="noopener noreferrer">  
    MDN Web Docs  
</a>
```

The `rel="noopener noreferrer"` attribute is not optional when using `target="_blank"`. Without it, the opened page can access the opener's window object via `window.opener` — a security vulnerability known as reverse tabnapping. Always pair `target="_blank"` with `rel="noopener noreferrer"`.

The CSS rule `list-style-position: inside` keeps the list numbers visually aligned within the content container rather than hanging outside it:

```
ol {  
    list-style-position: inside;  
}
```

4.6 The Contacts Page and HTML Forms

A contact form lets visitors send messages directly through the website. HTML provides all the elements needed — no JavaScript required for the structure.

The `<form>` tag is the container. Its two most important attributes are `action` (where to send the data) and `method` (how to send it). In this project `action` is empty because there is no backend server:

```
<form onsubmit="return false">

  <label for="name">Name:</label>
  <input type="text" id="name" name="name"
    placeholder="Your name" required>

  <label for="email">Email:</label>
  <input type="email" id="email" name="email"
    placeholder="Your email" required>

  <label for="message">Message:</label>
  <textarea id="message" name="message" rows="8"
    placeholder="Write your message here..."></textarea>

  <button type="submit">Send Message</button>

</form>
```

The key elements and their roles:

Element	Purpose
<code><label></code>	Associates a visible description with a field; clicking the label focuses the field
<code><input type="text"></code>	Single-line text field
<code><input type="email"></code>	Email field — browser validates the format automatically
<code><textarea></code>	Multi-line text field
<code><button type="submit"></code>	Submits the form

The `for` attribute on `<label>` must match the `id` attribute on the associated field. This pairing is essential for accessibility: screen readers announce the label when the field receives focus, and clicking the label moves focus to the field.

The `required` attribute activates browser-native validation — the form cannot be submitted if

the field is empty, without a single line of JavaScript.

`type="email"` is a free input validator: the browser rejects values that do not contain an @ sign and a domain. It is not a substitute for server-side validation, but it catches obvious mistakes immediately.

The form elements are styled in `page.css` to match the overall site look:

```
form label {
    display: block;
    margin-top: 10px;
    font-weight: bold;
    color: #444444;
}

form input,
form textarea {
    width: 100%;
    padding: 8px;
    border: 1px solid #cccccc;
    border-radius: 4px;
    box-sizing: border-box;
}

form button {
    margin-top: 12px;
    padding: 10px 24px;
    background-color: #1D67B2;
    color: #ffffff;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    font-weight: bold;
}
```

`box-sizing: border-box` is important: without it, `width: 100%` plus padding would make the field wider than its container. With `border-box`, the padding is included in the declared width, so the field stays within bounds.

4.7 The Gallery Page: CSS Grid and JavaScript

The gallery is the most technically interesting page in Part 3. It introduces two things: CSS Grid for the image grid layout, and a small JavaScript file for the lightbox viewer.

4.7.1 CSS Grid for the Image Grid

Flexbox is ideal for one-dimensional layouts — a row of icons, a column of cards. CSS Grid is designed for two dimensions: rows and columns simultaneously. A photo gallery is the perfect use case.

```
.gallery-grid {  
  display: grid;  
  grid-template-columns: repeat(auto-fill, minmax(150px, 1fr));  
  gap: 10px;  
  margin-top: 20px;  
}
```

The `repeat(auto-fill, minmax(150px, 1fr))` value is the key. Breaking it down:

- `auto-fill` — create as many columns as fit in the available width
- `minmax(150px, 1fr)` — each column is at least 150 px wide and at most one equal fraction of the available space

The result is a fully responsive grid: on a wide screen you get many columns; on a phone you might get two or three — with zero media queries.

Each cell clips the image to a fixed height:

```
.gallery-cell img {  
  width: 100%;  
  height: 150px;  
  object-fit: cover;  
  cursor: pointer;  
  opacity: 0.85;  
  transition: opacity 0.2s ease;  
}  
  
.gallery-cell img:hover {  
  opacity: 1;  
}
```

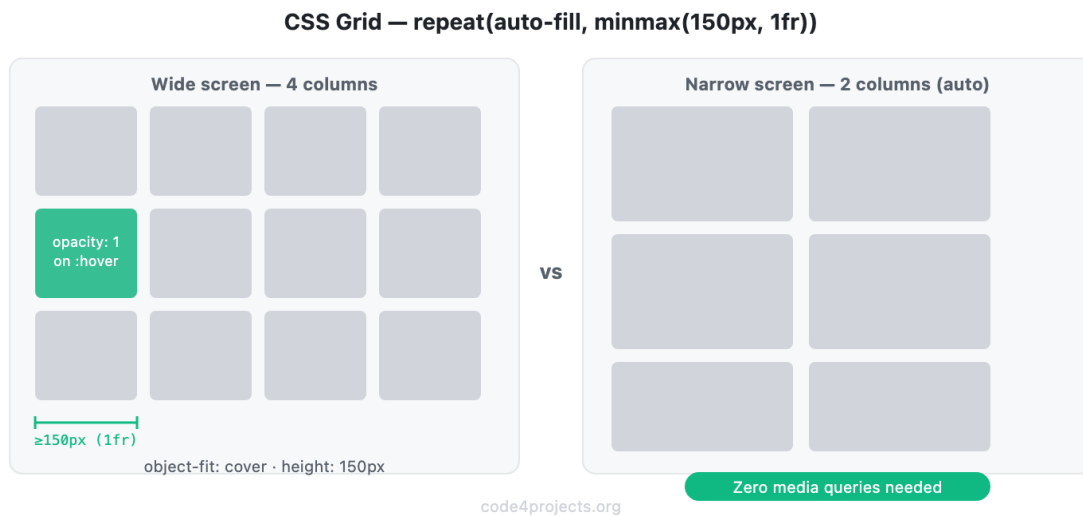


Figure 4.2: CSS Grid — auto-fill responsive layout

`object-fit: cover` scales the image to fill its cell without distortion, cropping the edges if necessary — the same behaviour as `background-size: cover` for background images.

4.7.2 The JavaScript Lightbox

When a user clicks a thumbnail, a full-screen viewer opens. This requires JavaScript because it involves responding to a user event and changing the DOM.

The script is kept in a separate file, `assets/js/gallery.js`, and loaded with the `defer` attribute:

```
<script type="text/javascript" src="assets/js/gallery.js" defer></script>
```

The `defer` attribute tells the browser to download the script in parallel with the HTML, but execute it only after the DOM is fully parsed. This prevents the classic error where a script tries to access an element that has not been created yet.

The gallery HTML uses an `onclick` attribute to call the JavaScript function:

```
<div class="gallery-cell">
  
</div>
```

The viewer is a hidden `<div>` that becomes visible when an image is clicked:

```
<div id="gallery-viewer">
  <span id="gallery-close" onclick="closeImage()">&#215;</span>
  <img id="gallery-expanded-img" src="" alt="">
</div>
```

The JavaScript is minimal — two functions:

```
function showImage(img) {
  var viewer = document.getElementById("gallery-viewer");
  var expandedImg = document.getElementById("gallery-expanded-img");
  expandedImg.src = img.src;
  expandedImg.alt = img.alt;
  viewer.style.display = "block";
}

function closeImage() {
  document.getElementById("gallery-viewer").style.display = "none";
}
```

`showImage` copies the `src` and `alt` from the clicked thumbnail into the expanded image, then makes the viewer visible by setting `style.display = "block"`. `closeImage` hides it again by setting `display = "none"`.

The viewer is styled as a fixed full-screen overlay:

```
#gallery-viewer {
  display: none;
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  background-color: rgba(0, 0, 0, 0.9);
  z-index: 100;
  text-align: center;
  padding-top: 50px;
  box-sizing: border-box;
}
```

`position: fixed` positions the viewer relative to the viewport, not the page — it stays in place even when the user scrolls. `z-index: 100` places it above everything else on the page.

4.8 The Final File Structure

After all Part 3 lessons, the project has grown from a layout shell to a complete multi-page site:

```
lesson-24/
├─ index.html
├─ about-me.html
├─ start-here.html
├─ resources.html
├─ contacts.html
├─ gallery.html
├─ assets/
│   ├─ css/
│   │   ├─ default.css      ← shared by all pages
│   │   ├─ home.css        ← home page only
│   │   ├─ page.css        ← inner pages, form, gallery
│   │   ├─ fontawesome.css ← Font Awesome core
│   │   └─ brands.css      ← Font Awesome brand icons
│   ├─ fontawesome/
│   │   └─ webfonts/       ← .ttf and .woff2 font files
│   ├─ img/               ← site images
│   ├─ gallery/           ← gallery photos
│   └─ js/
│       └─ gallery.js      ← gallery lightbox
```

The three-CSS-file architecture (`default.css`, `home.css`, `page.css`) keeps each stylesheet focused: shared structure, home-page layout, and inner-page content styles never mix. Adding a new page type in the future means adding a new CSS file, not modifying an existing one.

4.9 Conclusion

In this article we covered:

- How to introduce `page.css` to separate inner-page styles from home-page styles
- How to use floated images on article pages and remove the float on mobile with media queries

- How `<figure>` and `<figcaption>` communicate image-caption relationships to browsers and assistive technologies
- How `target="_blank"` combined with `rel="noopener noreferrer"` safely opens links in new tabs
- How to build a contact form with `<input>`, `<label>`, `<textarea>`, and `<button>`, and how browser-native validation works
- How CSS Grid with `repeat(auto-fill, minmax(...))` creates a fully responsive photo grid without media queries
- How to write a JavaScript lightbox in two functions and load it with `defer`

The series has now covered the full beginner path: from a blank HTML file to a complete, responsive, multi-page website with a form and an interactive gallery. This is the final article in the series.

Glossary

Attribute Extra information added to the opening tag of an HTML element, written as a name and a value, such as `href="index.html"` or `alt="Logo"`.

Box Model The CSS model that describes every element as a rectangular box made of four layers: content, padding, border, and margin.

Breakpoint The screen width at which a media query changes the layout of a page, for example switching from a horizontal menu to a hamburger menu.

Cascade The set of rules CSS uses to decide which declaration wins when several rules target the same element. It considers origin, specificity, and source order.

CSS (Cascading Style Sheets) The language that describes how HTML elements are displayed: colours, fonts, spacing, layout, and animations.

CSS Grid A two-dimensional CSS layout system that arranges elements in rows and columns. Ideal for galleries and page-level layouts.

Declaration A single property: value pair inside a CSS rule, such as `color: blue;`.

DOCTYPE The first line of an HTML document (`<!DOCTYPE html>`). It tells the browser which version of HTML the page uses, so it renders in standards mode.

Element The basic building block of an HTML page, made of an opening tag, content, and a closing tag, such as `<p>Hello</p>`.

Flexbox A one-dimensional CSS layout system that distributes space among items in a row or a column. Commonly used for navigation bars and headers.

Font Awesome A popular icon library that provides scalable vector icons (such as social media logos) through CSS classes.

Form An HTML element (`<form>`) that collects user input through controls such as `<input>`, `<textarea>`, and `<button>`.

Hamburger Menu The three-line icon that replaces a horizontal navigation bar on small screens and opens the menu when tapped.

Hero Image A large, prominent banner image at the top of a home page, usually combined with a short welcome message.

HTML (HyperText Markup Language) The standard markup language used to describe the structure and content of web pages.

Hyperlink A clickable reference to another page or location, created with the `<a>` tag and its `href` attribute.

Inheritance The CSS mechanism by which some properties, such as `color` and `font-family`, are passed from a parent element to its children.

Lightbox A UI pattern that shows an enlarged image in an overlay on top of the page, typically closed by clicking outside it.

Lorem Ipsum Placeholder text used in design and development to fill a layout before the real content is available.

Media Query A CSS rule (`@media`) that applies styles only when a condition is true, such as a maximum screen width. The foundation of responsive design.

Pseudo-element A CSS keyword such as `::before` or `::after` that styles a specific part of an element, or inserts decorative content, without extra HTML.

Responsive Design An approach to web design that makes pages adapt to any screen size, from phones to large desktop monitors.

Selector The part of a CSS rule that determines which HTML elements the rule applies to: element, class, ID, attribute, and more.

Semantic HTML The use of HTML tags that describe the meaning of their content, such as `<header>`, `<nav>`, `<main>`, and `<footer>`, instead of generic `<div>` tags.

Specificity The weight CSS assigns to a selector. When two rules conflict, the one with the higher specificity wins: ID selectors beat class selectors, which beat element selectors.

Tag A keyword enclosed in angle brackets, such as `<p>` or `</p>`, that marks the beginning or the end of an HTML element.

Viewport The visible area of a web page in the browser window. The viewport meta tag tells mobile browsers to match the page width to the device width.

What's Next?

You have reached the end of this book. If you have worked through each chapter, you now know how to:

- Write a well-structured HTML document with headings, paragraphs, links, lists, and images
- Style a page with CSS selectors, the box model, typography, and colours
- Organise a site with semantic tags such as `<header>`, `<nav>`, `<main>`, and `<footer>`
- Build a navigation bar, a hero section, and a consistent header and footer
- Make a site responsive with the viewport meta tag, media queries, and a hamburger menu
- Collect user input with HTML forms
- Lay out a photo gallery with CSS Grid and add your first lines of JavaScript

That is a complete, practical foundation. You have gone from a blank text file to a real, multi-page website that works on every screen.

Where to Go From Here

JavaScript

The lightbox in the last chapter was only a taste. JavaScript turns static pages into interactive applications: form validation, dynamic content, calls to web APIs. It is the natural third language of the web after HTML and CSS.

Modern CSS

CSS keeps evolving. Custom properties (CSS variables), container queries, the `:has()` selector, and native nesting let you write cleaner, more maintainable stylesheets with less code.

Accessibility

A website is only as good as the number of people who can use it. Learning ARIA attributes, keyboard navigation, and colour-contrast guidelines makes your pages usable by everyone, including people who rely on screen readers.

Static Site Generators and Hosting

Writing every page by hand does not scale. Tools such as Jekyll or Hugo generate a whole site from templates and Markdown files, and services such as GitHub Pages host it for free. This very blog, Code4Projects, is built that way.

Stay in Touch

If this book helped you, consider:

- Subscribing to the **Code4Projects newsletter** on Substack for new articles and future ebook releases
- Starring the GitHub repository at github.com/sasadangelo/html-hero
- Sharing the book with a friend who wants to build their first website

Thank you for reading.

— Salvatore D'Angelo

code4projects.org

*The best way to learn HTML and CSS is to build something.
Start with one page. Build from there.*